

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

ZAVRŠNI RAD br. 2252

**PLANIRANJE PUTANJE MOBILNOG ROBOTA U
POZNATOM PROSTORU PRIMJENOM
NELINEARNOG MODELSKOG PREDIKTIVNOG
UPRAVLJANJA**

Luka Kordić

Zagreb, lipanj, 2026.

ZAVRŠNI ZADATAK br. 2252

Pristupnik: **Luka Kordić (0036552456)**
Studij: Elektrotehnika i informacijska tehnologija i Računarstvo
Modul: Računarstvo
Mentor: izv. prof. dr. sc. Branimir Novoselnik

Zadatak: **Planiranje putanje mobilnog robota u poznatom prostoru primjenom nelinearnog modelskog prediktivnog upravljanja**

Opis zadatka:

U ovom završnom radu razmatra se problem planiranja i praćenja putanje mobilnog robota u poznatom i statičkom prostoru primjenom nelinearnog modelskog prediktivnog upravljanja (NMPC). Pretpostavlja se da je okruženje unaprijed definirano, uključujući granice prostora i eventualne statičke zapreke, bez potrebe za dinamičkim izbjegavanjem prepreka. Rad obuhvaća modeliranje mobilnog robota korištenjem prikladnog nelinearnog modela, formulaciju optimalnog upravljačkog problema s ograničenjima na upravljačke veličine i gibanje unutar dopuštenog prostora te implementaciju regulatora primjenom optimizacijskog alata acados. Razvit će se simulacijsko okruženje za rad u zatvorenoj petlji i provesti eksperimentalna evaluacija performansi sustava. Analizirat će se kvaliteta planirane i ostvarene putanje, vrijeme dolaska do cilja, poštivanje ograničenja i računalna složenost rješenja. Cilj rada je implementirati i razumjeti pristup nelinearnog modelskog prediktivnog upravljanja za planiranje i praćenje putanje te eksperimentalno procijeniti njegovu učinkovitost u simulacijskom okruženju.

Rok za predaju rada: 12. lipnja 2026.

*Zahvaljujem mentoru izv. prof. dr. sc. Branimiru Novoselnik na usmjeravanju, korisnim
komentarima i strpljenju tijekom izrade ovog rada.*

Sadržaj

1. Uvod	4
1.1. Motivacija i kontekst	4
1.2. Pregled pristupa planiranju i regulaciji	4
1.2.1. Planiranje putanje	4
1.2.2. Praćenje putanje	5
1.3. Doprinosi i opseg rada	5
2. Model mobilnog robota	7
2.1. Klasični kinematički model monocikla	7
2.2. Prošireni model: ubrzanja kao upravljačke veličine	8
2.3. Numerička integracija — metoda Runge-Kutta 4. reda	10
2.4. Veza s diferencijalnim pogonom	11
3. Nelinearno modelsko prediktivno upravljanje	12
3.1. Što je MPC	12
3.2. Formulacija optimizacijskog problema	14
3.3. Funkcija troška	15
3.4. Ograničenja	16
3.5. Meka ograničenja izbjegavanja prepreka	17
3.6. SQP_RTI solver	18
4. Globalno planiranje putanje	20
4.1. A* algoritam	20
4.1.1. Diskretizacija prostora	20
4.1.2. Pretraga	22
4.1.3. Pravokutne prepreke	24

4.2. Izgladivanje putanje	26
4.3. Profil brzine	27
5. Implementacija sustava	29
5.1. Arhitektura sustava	29
5.2. Simulacijska petlja zatvorene petlje	30
5.3. Metrike kvalitete praćenja	34
5.4. Testni scenariji	36
6. Rezultati i evaluacija	38
6.1. Eksperimentalni postav	38
6.2. Osnovna putanja	40
6.3. Jedna prepreka	42
6.4. Uski prolaz	45
6.5. L-hodnik	48
6.6. U-oblik	50
6.7. Poremećaj i oporavak	52
6.8. Gusti raspored prepreka	54
6.9. Analiza osjetljivosti: predikcijski horizont N	56
6.10. Analiza osjetljivosti: težine troška	58
6.11. Zbirna usporedba scenarija	59
6.12. Nedostižni cilj — granice sustava	59
7. Zaključak	61
7.1. Pregled postignutih rezultata	61
7.2. Analiza ključnih nalaza	61
7.3. Ograničenja implementiranog sustava	62
7.4. Smjernice za buduća istraživanja	63
7.5. Završna ocjena	64
Literatura	65
Izjava o korištenju umjetne inteligencije	67
Sažetak	68

Abstract 69

1. Uvod

1.1. Motivacija i kontekst

Robotski usisavači su rašireni primjer komercijalnih autonomnih sustava. Uređaji poput iRobot Roomba ili Ecovacs Deebot svakodnevno rješavaju zadatak autonomne navigacije u stambenom prostoru: kreću od baze za punjenje, pokrivaju čitavu površinu poda, izbjegavaju stolice, kućne ljubimce i stepenice, te se vraćaju na bazu kada završe ili im se isprazni baterija. Iza toga se, međutim, kriju dva klasična problema robotike: kako pronaći put od polazišta do cilja bez sudara s preprekama i kako robot stvarno prati taj put dovoljno precizno.

U ovom radu ta su dva problema riješena zajedno: izgrađen je sustav koji prvo planira put kroz prostor s preprekama, a zatim robot točno prati taj put. Koristi se model monocikla — jednostavan matematički opis kretanja robota koji dobro opisuje kako se kreće diferencijalno upravljani robot poput usisavača. Za razliku od problema poput uspravljanja njihala na kolicima gdje su sile i inercija neophodne za opis ponašanja sustava, za male brzine i ravnu podlogu kinematički model je dovoljna aproksimacija za potrebe ovog rada. Svi su eksperimenti provedeni u simulaciji u unaprijed poznatoj okolini.

1.2. Pregled pristupa planiranju i regulaciji

1.2.1. Planiranje putanje

Razvoj algoritama za planiranje putanje mobilnih robota traje desetljećima. Najraniji pristupi temelje se na pretraživanju grafova u diskretiziranom prostoru. Dijkstrin algoritam garantira optimalnost ali ne koristi heurističku informaciju o cilju. **A* algoritam** [1], uveden 1968. godine, dodaje procjenu preostale udaljenosti do cilja — ravnu liniju

do cilja — i time značajno ubrzava pretraživanje uz zadržavanje optimalnosti na diskretnoj rešetki [2]. A* je u širokoj primjeni, među ostalim u ROS-u (eng. *Robot Operating System*), standardnoj platformi istraživačke robotike.

1.2.2. Praćenje putanje

Problem praćenja putanje na prvi pogled izgleda jednostavno — robot treba slijediti zadanu liniju. No mobilni robot poput monocikla ima jedno ključno ograničenje: može se kretati samo u smjeru u koji je okrenut, ne i bočno. To znači da robot se ne može bočno pomaknuti na putanju ako se od nje odmakne, nego mora okretanjem i vožnjom naprijed postupno korigirati svoju poziciju i orijentaciju. Uz to, robot ima fizikalna ograničenja — maksimalnu brzinu, maksimalnu kutnu brzinu i ograničenja na to koliko brzo može mijenjati te veličine.

Za praćenje generirane putanje postoje razni pristupi — geometrijski regulatori (Pure Pursuit [3], Stanley), PID-regulatori i LQR. Svi su relativno jednostavni za implementaciju, ali zajednički im je nedostatak: ne gledaju unaprijed i ne uzimaju u obzir ograničenja sustava na formalan način.

Modelsko prediktivno upravljanje (eng. *Model Predictive Control*, MPC) [4, 5] rješava upravo to: na svakom koraku gleda unaprijed kroz cijeli predikcijski horizont i optimizira upravljanje uz eksplicitno poštivanje svih ograničenja. **Nelinearni MPC** (NMPC) proširuje taj pristup na nelinearne sustave poput monocikla. U ovom radu implementirano je korištenjem alata *acados* [6, 7].

1.3. Doprinosi i opseg rada

U ovom radu izgrađen je sustav koji kombinira A* algoritam za planiranje putanje s NMPC regulatorom za model monocikla. Cilj je bio razumjeti kako takav sustav radi u praksi i procijeniti učinkovitost takvog sustava u različitim situacijama.

Konkretno, u radu je napravljeno:

1. Prošireni model monocikla u kojemu su ubrzanja (a_v , α) upravljačke veličine, što olakšava postavljanje ograničenja u NMPC-u

2. Meka ograničenja izbjegavanja prepreka koja penaliziraju približavanje preprekama, ali ne uzrokuju neriješivost optimizacijskog problema
3. Simulacijsko okruženje s testiranjem robustnosti na vanjske poremećaje i vizualizacijom rezultata
4. Testiranje na sedam scenarija različite složenosti i analiza utjecaja ključnih parametara sustava

2. Model mobilnog robota

U ovom poglavlju opisuje se matematički model kretanja robota koji se koristi kroz cijeli rad. Važno je napomenuti da se radi o **kinematičkom** modelu — model opisuje *geometriju* kretanja (položaj, orijentacija, brzina) bez uzimanja u obzir masa, sila, momenata i inercije. Dinamički model, koji bi uključivao te efekte, bio bi znatno složeniji i izvan opsega ovog rada. Kinematički model je dovoljna aproksimacija za mobilnog robota koji se kreće po ravnoj podlozi pri malim brzinama.

Dodatno, model koristi apstrakciju **monocikla** (eng. *unicycle*): upravljačke veličine su translacijska i kutna brzina tijela robota (v, ω), a ne brzine pojedinih kotača (v_L, v_R). Ova apstrakcija je standardna u planiranju putanje i upravljanju mobilnim robotima; veza s fizičkim diferencijalnim pogonom opisana je u odjeljku 2.4.

2.1. Klasični kinematički model monocikla

U klasičnoj formulaciji monocikla [2] stanje robota opisano je s tri veličine:

$$\mathbf{x}_{\text{klas}} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \quad (2.1)$$

gdje su x i y koordinate robota u prostoru, a θ kut orijentacije (mjereno od osi x , od $-\pi$ do π). Kut θ je ciklička varijabla: kutovi θ i $\theta + 2\pi$ opisuju identičnu orijentaciju. U implementaciji se ograničenje $\theta \in [-\pi, \pi]$ eksplicitno postavlja kao stanjsko ograničenje solvera (hr. *rješavač*), a referentni kut računa se funkcijom `atan2` koja uvijek vraća vrijednost u istom rasponu. Time se izbjegava problem diskontinuiteta pri prijelazu $\theta = \pm\pi$, koji bi inače uzrokovao pogrešnu gradijentnu informaciju u kvadratnoj funkciji troška.

Upravljačke veličine su translacijska brzina v i kutna brzina ω :

$$\mathbf{u}_{\text{klas}} = \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (2.2)$$

Jednadžbe kretanja su:

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega \quad (2.3)$$

Iz ovih jednadžbi vidljiva je ključna nelinearnost: isti v daje potpuno drugačiji smjer promjene položaja ovisno o trenutnoj orijentaciji θ — ovo je i razlog zašto je potreban nelinearni regulator. Vidljiva je i neholonomska priroda monocikla: robot se može gibati samo u smjeru u koji je okrenut, ne i bočno. Ako se odmakne od referentne putanje, mora okretanjem i vožnjom postupno korigirati poziciju i orijentaciju.

2.2. Prošireni model: ubrzanja kao upravljačke veličine

U osnovnom modelu monocikla upravljačke veličine su direktno brzine v i ω . Međutim, u praksi robot ne može trenutačno promijeniti brzinu — postoje fizikalna ograničenja na to koliko brzo se brzina može mijenjati. Ako bismo ta ograničenja pokušali direktno implementirati u NMPC-u s v i ω kao upravljanjem, formulacija postaje komplicirana.

Rješenje je jednostavno: umjesto da upravljamo direktno brzinama, upravljamo translacijskim ubrzanjem $a_v = \dot{v}$ i kutnim ubrzanjem $\alpha = \dot{\omega}$. Brzine v i ω postaju dio vektora stanja, a upravljački vektor postaje:

$$\mathbf{u} = \begin{bmatrix} a_v \\ \alpha \end{bmatrix} \quad (2.4)$$

Cijeli vektor stanja sada ima pet komponenti, a dinamika proširenog modela glasi:

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ \omega \\ a_v \\ \alpha \end{bmatrix} \quad (2.5)$$

Prednost ovog pristupa je što fizikalna ograničenja na ubrzanje postaju jednostavna intervalna ograničenja na upravljanje:

$$|a_v| \leq 0,5 \text{ m/s}^2, \quad |\alpha| \leq 1,0 \text{ rad/s}^2 \quad (2.6)$$

A ograničenja na same iznose brzina postaju intervalna ograničenja na stanja:

$$0,05 \leq v \leq 1,0 \text{ m/s}, \quad |\omega| \leq 1,5 \text{ rad/s} \quad (2.7)$$

Minimalna brzina $v_{min} = 0,05 \text{ m/s}$ osigurava da robot uvijek napreduje prema cilju — bez te granice, optimizator bi u složenim situacijama mogao odabrati zaustavljanje kao lokalni optimum i robot bi zaostao iza referentne putanje koja kontinuirano napreduje.

Implementacija modela koristi CasADi [8, 9] — biblioteku za simboličku matematiku. Umjesto računanja s konkretnim brojevima, CasADi radi sa simboličkim varijablama poput x , θ , v iz kojih automatski izvodi derivacije potrebne za optimizaciju. To znači da dinamički model trebamo napisati samo jednom, a acados iz njega sam generira optimizirani C kod solvera bez ručnog računanja Jacobijana. Alternativa bi bila ručno izvođenje parcijalnih derivacija, no CasADi taj postupak čini bržim i jednostavnijim za promjene modela.

```

1 model.x = ca.vertcat(x, y, theta, v, omega)
2 model.u = ca.vertcat(dv, domega)
3
4 model.f_expl_expr = ca.vertcat(
5     v * ca.cos(theta),    # dx/dt
6     v * ca.sin(theta),    # dy/dt
7     omega,                # dtheta/dt
8     dv,                   # dv/dt
9     domega,               # domega/dt
10 )

```

Listing 1: Implementacija proširenog modela monocikla u `src/model.py`

2.3. Numerička integracija — metoda Runge-Kutta 4. reda

NMPC regulator interno rješava optimizacijski problem, ali nam je potrebna i odvojena numerička metoda za simulaciju "pravog robota" koji slijedi upravljačke naredbe. U tu svrhu koristi se metoda Runge-Kutta 4. reda (RK4) — standardna metoda numeričke integracije za sustave diferencijalnih jednadžbi [10].

RK4 aproksimira kretanje robota kroz jedan vremenski korak $\Delta t = 0,1$ s na sljedeći način:

$$k_1 = f(\mathbf{x}_k, \mathbf{u}_k) \quad (2.8)$$

$$k_2 = f\left(\mathbf{x}_k + \frac{\Delta t}{2}k_1, \mathbf{u}_k\right) \quad (2.9)$$

$$k_3 = f\left(\mathbf{x}_k + \frac{\Delta t}{2}k_2, \mathbf{u}_k\right) \quad (2.10)$$

$$k_4 = f(\mathbf{x}_k + \Delta t k_3, \mathbf{u}_k) \quad (2.11)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (2.12)$$

RK4 je odabran kao standardna metoda numeričke integracije koja se široko koristi u simulacijama dinamičkih sustava. Tijekom razvoja testirana je i Eulerova metoda, no

rezultati su bili gotovo identični RK4-u pa je u konačnoj implementaciji zadržan samo RK4. Implementacija u `src/simulation.py` direktno odgovara gornjim jednadžbama i koristi isti f kao i model definiran u `src/model.py`, čime je osigurana konzistentnost između simulacije i predikcijskog modela u NMPC-u.

2.4. Veza s diferencijalnim pogonom

Apstrakcija monocikla opisuje kretanje upravljačkim veličinama (v, ω) , ali fizički diferencijalno upravljani robot (dva neovisno pogonjena kotača) nema direktne aktutore za v i ω — pokretački signali su brzine pojedinih kotača (v_L, v_R) . Veza između ova dva opisa glasi [11]:

$$v = \frac{v_R + v_L}{2}, \quad \omega = \frac{v_R - v_L}{L} \quad (2.13)$$

gdje je L razmak između kotača (eng. *wheelbase*). Inverzna transformacija, potrebna za naredbu motorima:

$$v_R = v + \frac{\omega L}{2}, \quad v_L = v - \frac{\omega L}{2} \quad (2.14)$$

U ovom radu ta transformacija nije implementirana jer je sustav u potpunosti simulacijski — NMPC izračunava (v, ω) koji se direktno koriste kao ulaz u simulacijski model. Na stvarnom diferencijalnom robotu ova transformacija bila bi neophodna kao zadnji korak između izlaza regulatora i naredbi motornih kontrolera, uz poznavanje parametra L .

3. Nelinearno modelsko prediktivno upravljanje

3.1. Što je MPC

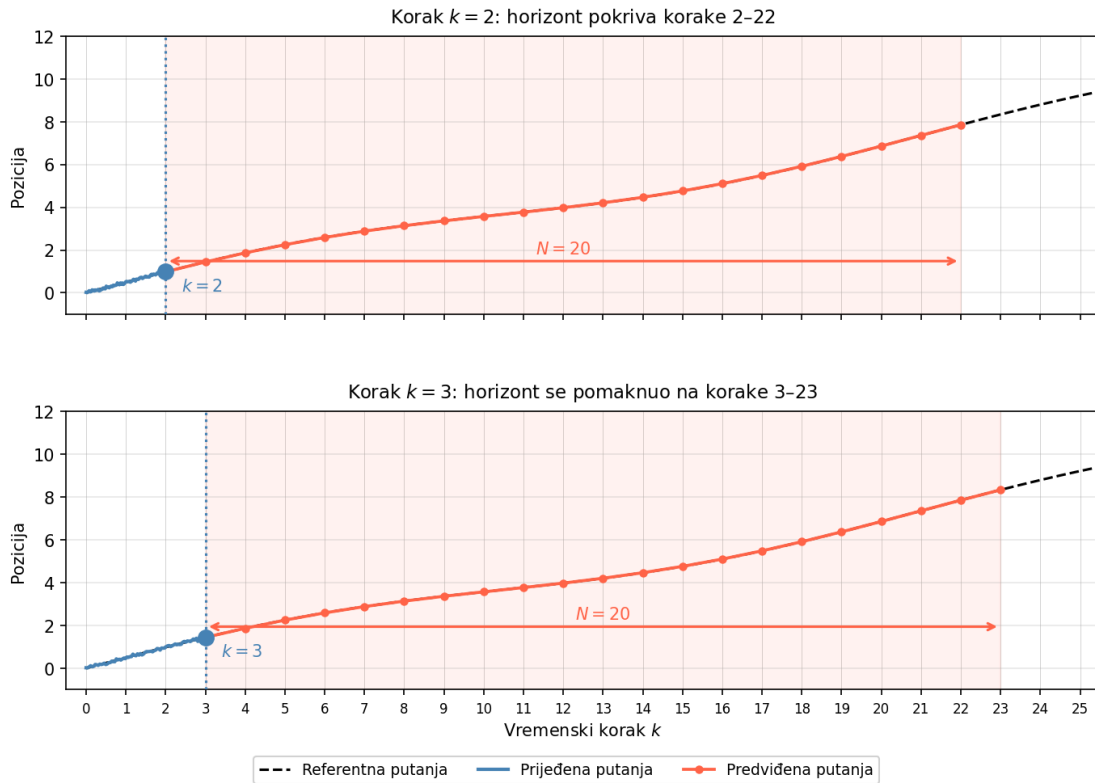
MPC (eng. *Model Predictive Control*) [4, 5] na svakom koraku predviđa ponašanje sustava za sljedećih N koraka i odabire upravljanje koje minimizira grešku u tom budućem prozoru, uz poštivanje svih ograničenja.

Analogno iskusnom vozaču koji gleda cestu 20 metara unaprijed: ne samo da uočava ono što dolazi, nego već sada planira buduće kočenje ili skretanje da problem riješi prije nego se pojavi.

Konkretno, na svakom koraku:

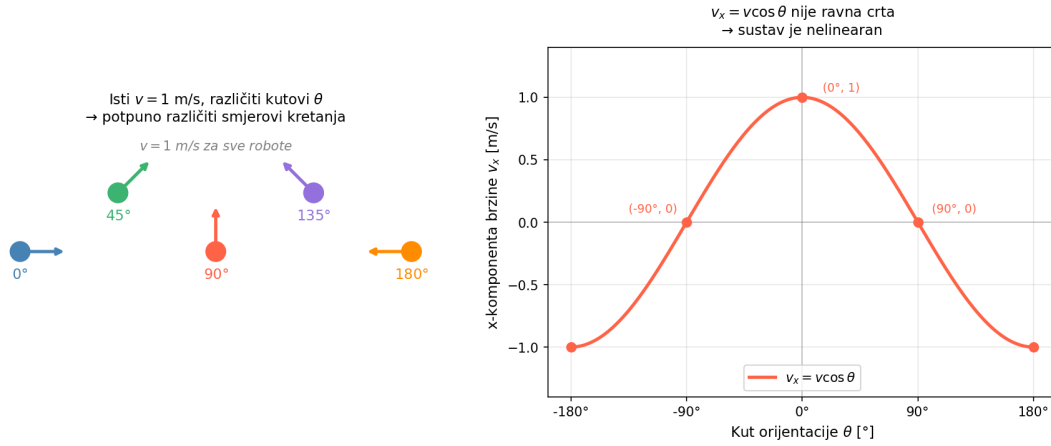
1. izmjeri se trenutno stanje robota $(x, y, \theta, v, \omega)$
2. optimizira se upravljanje za sljedećih $N = 20$ koraka (= 2 sekunde unaprijed)
3. primijeni se samo **prvi** upravljački signal iz tog plana
4. ponovi se u sljedećem koraku s novim mjerenjem

Ovaj pristup se naziva klizni horizont (eng. *receding horizon*): horizont se stalno pomiče naprijed zajedno s robotom. Prednost je što svako novo mjerenje automatski ispravlja akumulirane greške ili vanjske poremećaje.



Slika 3.1. Ilustracija kliznog horizonta. U koraku $k = 2$ NMPC optimizira upravljanje za korake 2–22 (crveni segment, $N = 20$). U sljedećem koraku $k = 3$ horizont se pomiče naprijed na korake 3–23. Plava linija pokazuje prijedenu putanju, a isprekidana crna referentnu.

Nelinearni MPC (NMPC) je varijanta koja radi direktno s nelinearnim modelom. Jednokolica je nelinearna jer njezine jednadžbe kretanja sadrže $v \cos \theta$ i $v \sin \theta$ — brzina i kut orijentacije su množeni, što sustav čini nelinearnim. Konkretno: robot koji vozi brzinom $v = 1$ m/s ravno ($\theta = 0$) kreće se isključivo u smjeru osi x , a isti robot koji vozi istom brzinom ali okrenut za $\theta = \pi/2$ kreće se isključivo u smjeru osi y — isti ulaz, potpuno drugačiji efekt ovisno o trenutnom stanju. Da je sustav linearan, način na koji upravljanje utječe na promjenu stanja bio bi konstantan i neovisan o trenutnoj orijentaciji. U monociklu naprotiv, faktori $\cos \theta$ i $\sin \theta$ množe brzinu v — isti upravljački signal pri različitom θ daje potpuno drugačiji smjer promjene pozicije. Zbog toga je potreban nelinearni optimizacijski pristup poput SQP-a.



Slika 3.2. Lijevo: robot pri istoj brzini $v = 1$ m/s ali različitim kutovima orijentacije — smjer kretanja potpuno se mijenja ovisno o stanju. Desno: x-komponenta brzine $v_x = v \cos \theta$ u ovisnosti o kutu — valovita krivulja, ne ravna crta, što znači da je sustav nelinearan.

3.2. Formulacija optimizacijskog problema

Cijeli NMPC problem može se zapisati kao optimizacijski problem s ograničenjima (OCP, eng. *Optimal Control Problem*):

$$\begin{aligned}
 \min_{\substack{\mathbf{u}_0, \dots, \mathbf{u}_{N-1} \\ \mathbf{s}_0, \dots, \mathbf{s}_{N-1}}} & \sum_{k=0}^{N-1} \|\mathbf{y}_k - \mathbf{y}_{ref,k}\|_W^2 + \|\mathbf{y}_N - \mathbf{y}_{ref,N}\|_{W_e}^2 + \sum_{k=0}^{N-1} \sum_i (w_L s_{i,k} + w_Q s_{i,k}^2) \\
 \text{uz uvjet} & \quad \mathbf{x}_{k+1} = \Phi(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \dots, N-1 \\
 & \quad \mathbf{x}_0 = \mathbf{x}_{meas} \\
 & \quad |a_{v,k}| \leq 0,5 \text{ m/s}^2, \quad |\alpha_k| \leq 1,0 \text{ rad/s}^2 \\
 & \quad 0,05 \leq v_k \leq 1,0 \text{ m/s}, \quad |\omega_k| \leq 1,5 \text{ rad/s} \\
 & \quad h_i(\mathbf{x}_k) + s_{i,k} \geq 0, \quad s_{i,k} \geq 0
 \end{aligned} \tag{3.1}$$

gdje $\Phi(\mathbf{x}_k, \mathbf{u}_k)$ označava numerički integrator dinamike s korakom $\Delta t = 0,1$ s (konkretno ERK4 (eng. *Explicit Runge-Kutta*, ERK) — eksplicitna Runge-Kutta metoda 4. reda), $h_i(\mathbf{x}) = \|\mathbf{p} - \mathbf{c}_i\|_2 - r_i$ je udaljenost centra robota $\mathbf{p} = (x, y)$ od granice prepreke i (centar $\mathbf{c}_i$, polumjer r_i), a $s_{i,k} \geq 0$ su slack varijable koje dopuštaju meka kršenja ograničenja prepreka uz kaznu $w_L = w_Q = 10^4$.

Sljedeće sekcije opisuju detaljno svaki od sastavnih dijelova ovog problema.

3.3. Funkcija troška

Na svakom koraku NMPC traži upravljačku sekvencu $\mathbf{u}_0, \dots, \mathbf{u}_{N-1}$ koja minimizira kvadratnu funkciju troška:

$$\min_{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}} \sum_{k=0}^{N-1} \|\mathbf{y}_k - \mathbf{y}_{ref,k}\|_W^2 + \|\mathbf{y}_N - \mathbf{y}_{ref,N}\|_{W_e}^2 \quad (3.2)$$

gdje je $\mathbf{y}_k$ izlazni vektor koji kombinira stanja i upravljanje, a $\mathbf{y}_{ref,k}$ referentni vektor koji dolazi s A* putanje:

$$\mathbf{y}_k = \begin{bmatrix} x_k \\ y_k \\ \theta_k \\ a_{v,k} \\ \alpha_k \end{bmatrix}, \quad \mathbf{y}_{ref,k} = \begin{bmatrix} x_{ref} \\ y_{ref} \\ \theta_{ref} \\ 0 \\ 0 \end{bmatrix} \quad (3.3)$$

Nule na kraju $\mathbf{y}_{ref,k}$ znače da su željena ubrzanja nula — robot treba mijenjati brzinu samo koliko je nužno za praćenje putanje. Matrica troška W blok-dijagonalne je strukture:

$$W = \begin{bmatrix} Q & 0 \\ 0 & R \end{bmatrix}, \quad Q = \begin{bmatrix} 10,0 & 0 & 0 \\ 0 & 10,0 & 0 \\ 0 & 0 & 1,0 \end{bmatrix}, \quad R = \begin{bmatrix} 0,1 & 0 \\ 0 & 0,1 \end{bmatrix} \quad (3.4)$$

Omjer dijagonalnih elemenata $Q_{xx}/R_{a_v} = 10,0/0,1 = 100$ znači da je korekcija pozicije 100 puta važnija od glatkoće upravljanja. Terminalna matrica $W_e = Q$ kažnjava grešku na kraju horizonta istim težinama — odabir $W_e = Q$ je heuristički i u praksi se pokazao stabilnim, no postoje i sofisticiranije metode (npr. rješavanje Riccatijeve jednadžbe).

Ove vrijednosti nisu odabrane nasumično — u poglavlju 6. prikazana je analiza kako različiti omjeri Q/R utječu na preciznost praćenja.

Izlazni vektor $\mathbf{y}_k$ dobiva se linearnom projekcijom stanja i upravljanja: $\mathbf{y}_k = V_x \mathbf{x}_k + V_u \mathbf{u}_k$, gdje su projekcijske matrice:

$$V_x = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad V_u = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \quad (3.5)$$

Gornje tri komponente izlaza (x, y, θ) dolaze iz vektora stanja, a donje dvije (a_v, α) iz vektora upravljanja. Odgovarajuća implementacija u acadosu:

```

1  Q = np.diag(mpc["Q"])    # diag(10, 10, 1) -- x, y, theta
2  R = np.diag(mpc["R"])    # diag(0.1, 0.1) -- dv, domega
3
4  ocp.cost.Vx = np.zeros((ny, nx))
5  ocp.cost.Vx[:, :3] = np.eye(3)    # uzmi x, y, theta iz stanja
6
7  ocp.cost.Vu = np.zeros((ny, nu))
8  ocp.cost.Vu[3:, :] = np.eye(nu)   # uzmi dv, domega iz upravljanja
9
10 ocp.cost.W   = np.block([[Q, np.zeros((3, nu))],
11                           [np.zeros((nu, 3)), R]])
12 ocp.cost.W_e = Q                # terminalna matrica = Q

```

Listing 2: Postavljanje matrica troška u `src/mpc_solver.py`

3.4. Ograničenja

Sva fizikalna ograničenja robota direktno su ugrađena u optimizacijski problem. Ograničenja na upravljanje (maksimalna ubrzanja):

$$|a_v| \leq 0,5 \text{ m/s}^2, \quad |\alpha| \leq 1,0 \text{ rad/s}^2 \quad (3.6)$$

Ograničenja na stanja (fizikalne granice robota i okolina):

$$0,05 \leq v \leq 1,0 \text{ m/s}, \quad |\omega| \leq 1,5 \text{ rad/s} \quad (3.7)$$

$$0 \leq x \leq 10 \text{ m}, \quad 0 \leq y \leq 10 \text{ m} \quad (3.8)$$

Implementacija u `src/mpc_solver.py`:

```

1 ocp.constraints.lbu = np.array([-robot["dv_max"], -robot["domega_max"]]) # dv,
  ↳ domega min
2 ocp.constraints.ubu = np.array([ robot["dv_max"],  robot["domega_max"]]) # dv,
  ↳ domega max
3
4 ocp.constraints.lbx = np.array([env["x_min"], env["y_min"], -np.pi,
5                               robot["v_min"], -robot["omega_max"]]) #
  ↳ donje granice stanja
6 ocp.constraints.ubx = np.array([env["x_max"], env["y_max"],  np.pi,
7                               robot["v_max"],  robot["omega_max"]]) #
  ↳ gornje granice stanja

```

Listing 3: Postavljanje ograničenja u `src/mpc_solver.py`

3.5. Meka ograničenja izbjegavanja prepreka

Za svaku kružnu prepreku s centrom (c_x, c_y) i polumjerom r definira se uvjet sigurnosne udaljenosti:

$$h(\mathbf{x}) = (x - c_x)^2 + (y - c_y)^2 - (r + r_{rob})^2 \geq 0 \quad (3.9)$$

gdje je $r_{rob} = 0,2 \text{ m}$ polumjer robota. Ovaj uvjet mora biti zadovoljen za svaki korak horizonta.

Ograničenja su implementirana kao **meka** (eng. *soft*): umjesto da se zahtijeva $h_i(\mathbf{x}_k) \geq 0$, uvodi se slack varijabla $s_{i,k} \geq 0$ i relaksirani uvjet postaje:

$$h_i(\mathbf{x}_k) + s_{i,k} \geq 0, \quad s_{i,k} \geq 0 \quad (3.10)$$

Kada je $s_{i,k} = 0$, ograničenje je zadovoljeno; kada je $s_{i,k} > 0$, robot je unutar sigurnosne zone prepreke i i veličina $s_{i,k}$ mjeri iznos kršenja. Svaka slack varijabla kažnjava se u funkciji troška linearnim i kvadratnim penalom:

$$\sum_{k=0}^{N-1} \sum_i (w_L s_{i,k} + w_Q s_{i,k}^2), \quad w_L = w_Q = 10^4 \quad (3.11)$$

Linearna komponenta pokreće robot iz prepreke čak i kod malih kršenja, a kvadratna brzo raste s većim kršenjima. Meka formulacija korisna je jer održava numeričku rješivost OCP-a u uskim prostorima i pri nesavršenim referencama — tvrda ograničenja bi u takvim situacijama uzrokovala neriješivost solvera.

```

1  for (cx, cy, r) in obstacles:
2      # h(x) = (x-cx)^2 + (y-cy)^2 - (r + r_robot)^2 >= 0
3      h_list.append((px - cx)**2 + (py - cy)**2 - (r + r_robot)**2)
4
5  ocp.model.con_h_expr = ca.vertcat(*h_list)    # nelinearna ograničenja za acados
6
7  penalty = 1e4                                # kazna za kršenje ograničenja
8  ocp.cost.zl = penalty * np.ones(n_obs)       # linearni penalty
9  ocp.cost.Zl = penalty * np.ones(n_obs)       # kvadratni penalty

```

Listing 4: Implementacija mekih ograničenja prepreka u `src/mpc_solver.py`

3.6. SQP_RTI solver

Za rješavanje nelinearnog optimizacijskog problema na svakom koraku koristi se SQP_RTI (eng. *Sequential Quadratic Programming — Real-Time Iteration*) pristup unutar alata acados [6, 7, 12].

SQP (eng. *Sequential Quadratic Programming*) je metoda optimizacije koja iterativno linearizira nelinearni problem i rješava niz kvadratičnih potprograma. SQP_RTI (eng. *Real-Time Iteration*) je varijanta prilagođena za primjene u stvarnom vremenu i sastoji se od dvije faze koje se izmjenjuju u svakom regulacijskom koraku [12]:

1. **Faza pripreme** (eng. *preparation phase*): linearizacija dinamike duž trenutne tra-

jektorije, izračun Jacobijana, kondenzacija kvadratičnog potprograma (eng. *Quadratic Program*, QP) — ove operacije odvijaju se *unaprijed*, dok robot još izvršava prethodni upravljački signal.

2. **Faza povratne veze** (eng. *feedback phase*): čim stigne novo mjerenje $\mathbf{x}_{meas}$, postavi se inicijalni uvjet i izvrši jedna SQP iteracija rješavanjem pripremljenog QP-a. Rezultat je novi upravljački signal koji se odmah šalje robotu.

Zahvaljujući ovoj podjeli, kašnjenje između mjerenja i primjene upravljanja iznosi samo trajanje feedback faze (najmanji dio ukupnog računanja), dok se skuplja priprema odvija paralelno s izvođenjem prethodnog koraka.

Za rješavanje kvadratičnog potprograma unutar svake SQP iteracije koristi se HPIPM rješavač (eng. *High Performance Interior Point Method*) koji je optimiziran za strukturu MPC problema. Gauss-Newtonova aproksimacija hesijana zamjenjuje pravi hesijan Lagrangiana aproksimacijom $J^T W J$, gdje je J Jacobijan reziduala (razlika $\mathbf{y}_k - \mathbf{y}_{ref,k}$) kvadratne funkcije troška, a W težinska matrica. Time se izbjegava skupo računanje drugih derivacija nelinearne dinamike i ograničenja uz cijenu blago slabije konvergencije — za RTI shemu s jednom iteracijom po koraku to je prihvatljiv kompromis.

Ukupno vrijeme rješavanja po koraku izmjereno u eksperimentima iznosi između 0,09 i 0,12 ms — daleko ispod vremenskog koraka od 100 ms, što ostavlja dovoljno prostora za opterećenje komunikacije i perifernih operacija.

```
1 ocp.solver_options.nlp_solver_type = mpc["solver_type"]    # "SQP_RTI"
2 ocp.solver_options.qp_solver       = "PARTIAL_CONDENSING_HPIPM"
3 ocp.solver_options.hessian_approx  = "GAUSS_NEWTON"
4 ocp.solver_options.integrator_type = "ERK"                # eksplicitni
  ↪ Runge-Kutta 4. reda
5 ocp.solver_options.nlp_solver_max_iter = 50
```

Listing 5: Konfiguracija SQP_RTI solvera u `src/mpc_solver.py`

4. Globalno planiranje putanje

4.1. A* algoritam

Prije nego NMPC može pratiti putanju, potrebno je tu putanju pronaći. A* algoritam [1, 2] pretražuje diskretiziranu mrežu prostora i pronalazi najkraći slobodan put od starta do cilja.

4.1.1. Diskretizacija prostora

Okolina dimenzija 10×10 m podijeli se u pravokutnu mrežu s rezolucijom $\delta = 0,2$ m po ćeliji, što daje mrežu $51 \times 51 = 2601$ čvorova. Svaka ćelija označena je kao slobodna ili blokirana na osnovu udaljenosti od prepreke:

$$\text{blokirana}(i, j) = \|w_{i,j} - c_k\|_2 \leq r_k + r_{rob} + \delta_{inf} \quad (4.1)$$

gdje je $w_{i,j}$ koordinata ćelije u prostoru, c_k centar prepreke, r_k njezin polumjer, $r_{rob} = 0,2$ m polumjer robota i $\delta_{inf} = 0,2$ m dodatno proširenje prepreke. Ukupno proširenje prepreke iznosi $r_{rob} + \delta_{inf} = 0,4$ m — to znači da se centar robota ne može naći unutar 0,4 m od ruba bilo koje prepreke.

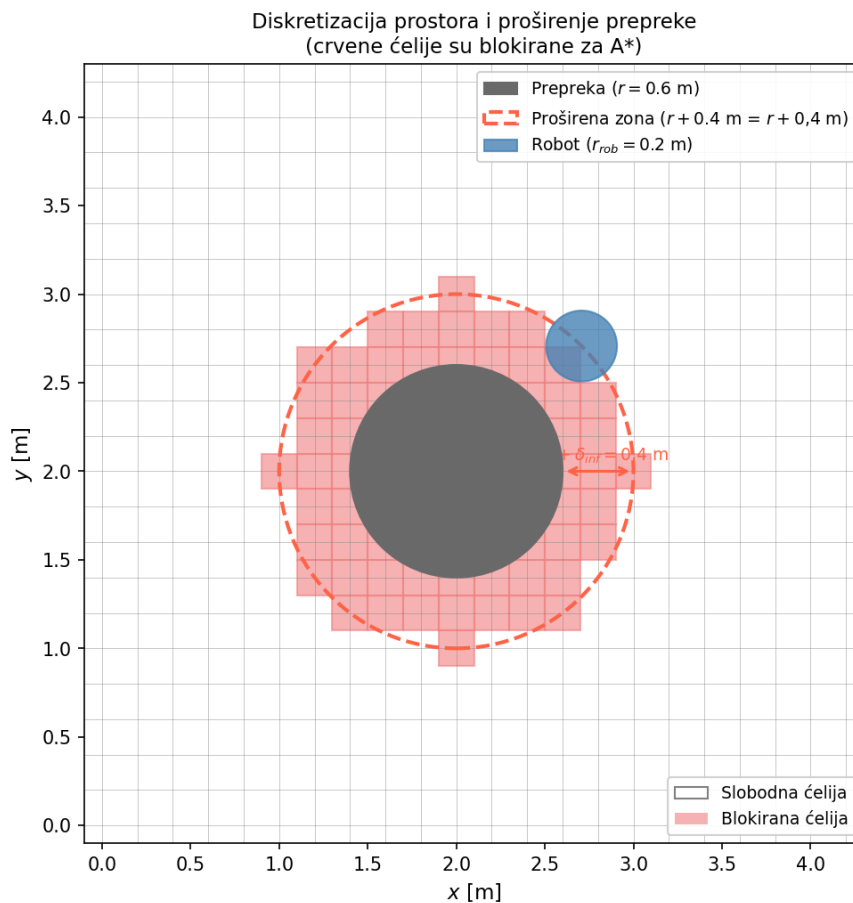
Implementacija u `src/astar.py`:

```

1  # ukupno proširenje = sigurnosna margina + polumjer robota
2  inflation = p["astar"]["obstacle_inflation"] + p["robot"]["radius"] # 0.2 + 0.2
   ↳ = 0.4 m
3
4  for (cx, cy, r) in to_acados_list(obstacles): # za svaku prepreku (centar +
   ↳ polumjer)
5      for i in range(nx):
6          for j in range(ny):
7              wx, wy = grid_to_world(i, j) # indeks ćelije → koordinata u
   ↳ prostoru
8              # ćelija je blokirana ako joj je centar unutar proširene prepreke
9              if (wx - cx)**2 + (wy - cy)**2 <= (r + inflation)**2:
10                 occupied[i, j] = True

```

Listing 6: Označavanje blokiranih ćelija u `src/astar.py`



Slika 4.1. Diskretizacija prostora i proširenje prepreke. Crvene ćelije su blokirane za A* jer im je centar unutar proširene zone (isprekidani krug). Robot (plavi krug) može se kretati samo centrom slobodnih ćelija, što osigurava da A* putanja ostaje izvan sigurnosne zone prepreke.

4.1.2. Pretraga

A* koristi prioritetni red i za svaki čvor n prati procjenu ukupnog troška:

$$f(n) = g(n) + h(n) \quad (4.2)$$

gdje je $g(n)$ stvarni trošak puta od starta do čvora n , a $h(n)$ euklidska udaljenost do cilja:

$$h(n) = \sqrt{(n_x - g_x)^2 + (n_y - g_y)^2} \quad (4.3)$$

Algoritam uvijek proširuje čvor s najmanjim $f(n)$. Euklidska heuristika nikad ne precjenjuje stvarni trošak, što garantira pronalazak najkraćeg puta na diskretnoj rešetki. Susjedi se razmatraju u 8 smjerova — 4 kardinalna (trošak 1,0) i 4 dijagonalna (trošak $\sqrt{2}$).

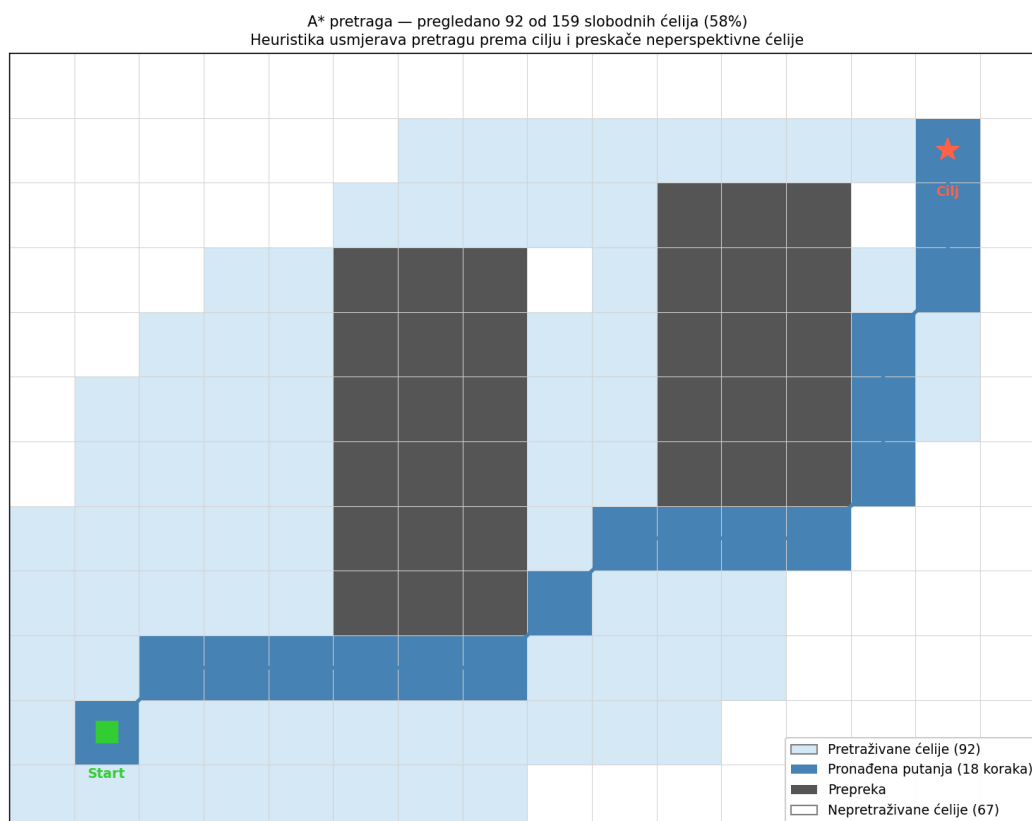
Kada A* prođe kroz sve dostupne ćelije i ne pronade put do cilja, program javlja grešku — to znači da put ne postoji:

```

1  neighbours = [(-1,0),(1,0),(0,-1),(0,1),      # kardinalni smjerovi
2              (-1,-1),(-1,1),(1,-1),(1,1)]      # dijagonalni smjerovi
3
4  while open_set:
5      _, g, current = heapq.heappop(open_set)      # uzmi čvor s najmanjim f
6      if current == goal_g:
7          # rekonstruiraj put unatrag
8          ...
9      for di, dj in neighbours:
10         step = np.hypot(di, dj)                  # 1.0 ili sqrt(2)
11         new_g = g + step
12         if new_g < g_score.get(nb, np.inf):
13             f = new_g + heuristic(nb, goal_g)
14             heapq.heappush(open_set, (f, new_g, nb))
15
16  raise RuntimeError("A*: nema puta od starta do cilja")

```

Listing 7: Jezgra A* pretrage u src/astar.py



Slika 4.2. Vizualizacija A* pretrage na primjeru mreže s dvije prepreke. Svjetloplavi čvorovi su pretraživani, tamno plavi čvorovi su na pronađenoj putanji, bijeli čvorovi nisu ni razmatrani. Heuristika usmjerava pretragu prema cilju i preskače neperspektivne čvorove.

4.1.3. Pravokutne prepreke

A* radi s kružnim preprekama jer za njih proširenje prepreke ima jasnu geometrijsku interpretaciju. Pravokutne prepreke (korištene u L-hodniku i U-obliku) aproksimiraju se mrežom malih kružnica polumjera 0,2 m, što je razlog opisan u poglavlju 3. — acados zahtijeva glatke, derivabilne funkcije za računanje Jacobijana.

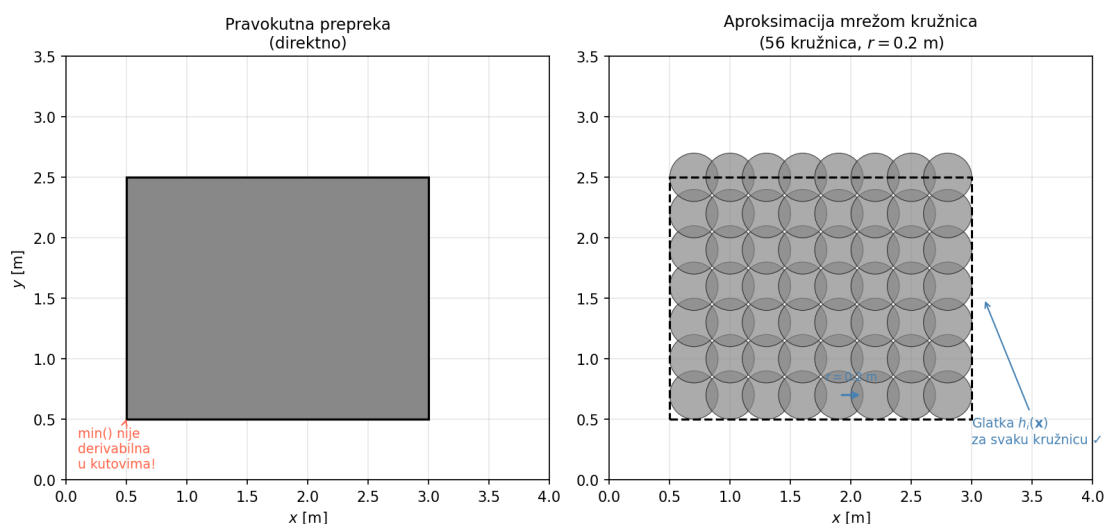
```

1 def to_circles(self, r: float = 0.2) -> list[Circle]:
2     circles = []
3     x = self.x_min + r          # počni od lijevog ruba + polumjer
4     while x <= self.x_max:
5         y = self.y_min + r      # počni od donjeg ruba + polumjer
6         while y <= self.y_max:
7             circles.append(Circle(x, y, r)) # dodaj kružnicu u točki (x, y)
8             y += r * 1.5        # razmak između kružnica (dovoljno gusto da
                                ↪ nema prolaza)
9         x += r * 1.5
10    return circles

```

Listing 8: Aproksimacija pravokutnika mrežom kružnica u `src/obstacles.py`

Razmak između centara susjednih kružnica iznosi $1,5 \cdot r = 0,3$ m, pa je razmak između rubova susjednih kružnica $0,3 - 2 \cdot 0,2 = -0,1$ m — kružnice se preklapaju za 0,1 m, što garantira da ne postoji prolaz kroz koji bi mogao proći robot polumjera $r_{rob} = 0,2$ m. Za scenarij L-hodnika ova shema daje **525 kružnica**, a za U-oblik (3 pravokutnika) **162 kružnice**. Taj broj postaje broj nelinearnih ograničenja prepreka u OCP-u, što utječe na CPU vrijeme solvera.



Slika 4.3. Lijevo: pravokutna prepreka direktno — ograničenje $h(\mathbf{x})$ nije glatko u kutovima pa acados ne može računati Jacobijane. Desno: ista prepreka aproksimirana mrežom kružnica — svaka kružnica ima glatku $h_i(\mathbf{x})$ pa SQP iteracije rade ispravno.

4.2. Izgladivanje putanje

A* putanja sadrži oštre kutove jer je ograničena rešetkom. Takva putanja nije pogodna kao referenca za NMPC jer zahtijeva nagle promjene smjera. Stoga se izgladuje **kubnim B-splineom** koristeći funkciju `splprep` iz biblioteke SciPy [13]:

```
1 tck, _ = splprep([pts[:, 0], pts[:, 1]], s=0.5, k=3) # kubični spline, s=0.5
2 t = np.linspace(0, 1, n_points)                    # 200 jednakomjernih
   ↪ točaka
3 x_s, y_s = splev(t, tck)
```

Listing 9: Izgladivanje putanje kubičnim splineom u `src/path_smoother.py`

Parametar izgladivanja $s = 0,5$ dopušta B-spline krivulji da ne prolazi točno kroz sve A* točke, nego pronalazi gladu krivu koja eliminira sitne oscilacije rešetke. Rezultat je 200 ravnomjerno raspoređenih točaka duž glatke krivulje.

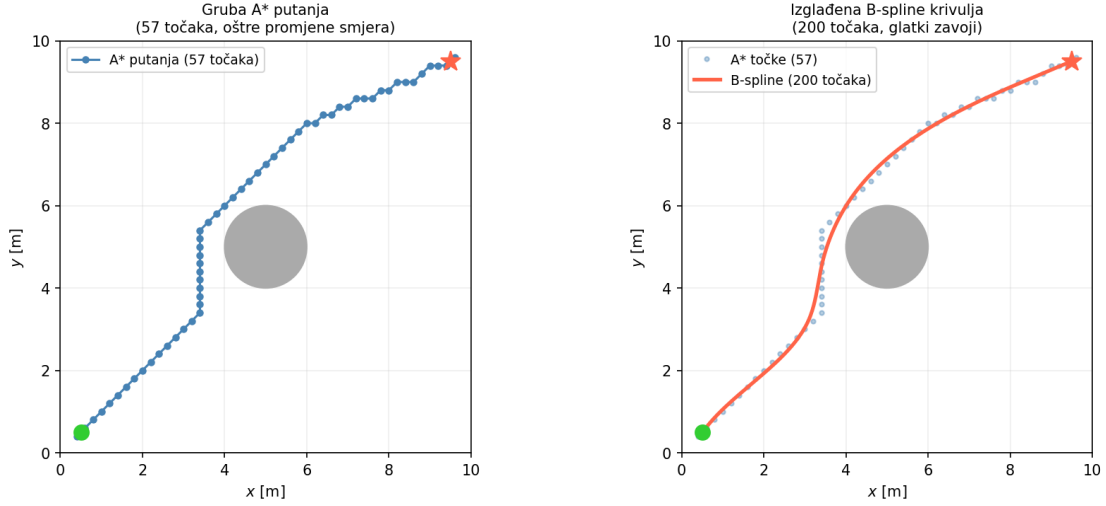
Budući da B-spline s $s > 0$ ne prolazi točno kroz A* točke, izgladena krivulja može se teoretski primaknuti prepreci bliže od izvorne diskretne putanje. Zbog toga se nakon izgladivanja provjerava minimalna udaljenost svake od 200 točaka od svih prepreka. Ako je minimalna udaljenost manja od $r_{rob} = 0,2$ m, parametar s se smanjuje i izgladivanje se ponavlja (redoslijed: $s = 0,5 \rightarrow 0,3 \rightarrow 0,1 \rightarrow 0,0$); pri $s = 0$ spline interpolira točno kroz A* točke koje su po konstrukciji u sigurnoj zoni.

U svim testiranim scenarijima prvi prolaz s $s = 0,5$ prošao je bez kršenja. Minimalne izmjerene udaljenosti izgladene putanje od ruba prepreka prikazane su u tablici 4.1.

Tablica 4.1. Minimalna udaljenost izgladene B-spline putanje od ruba prepreke po scenariju

Scenarij	Min. udaljenost [m]
one_obstacle	0,357
narrow	0,348
l_corridor	0,229
u_shape	0,304
cluttered	0,296

Najmanji razmak postiže se u scenariju L-hodnika (0,229 m), gdje B-spline "reže" unutarnji kut zida — no i ta vrijednost premašuje polumjer robota $r_{rob} = 0,2$ m, pa u eksperimentima nije zabilježeno narušavanje sigurnosne udaljenosti.



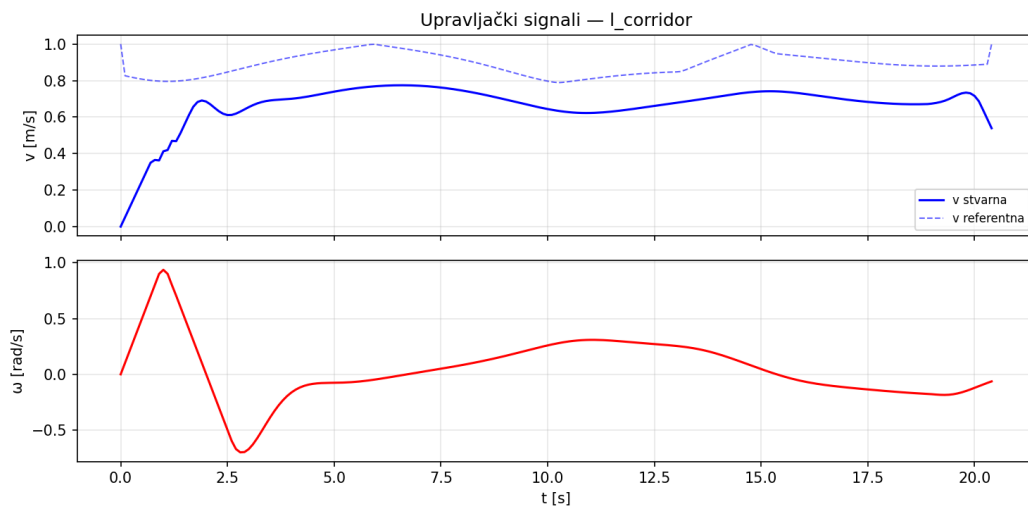
Slika 4.4. Lijevo: gruba A* putanja s 57 točaka — vidljive su oštre promjene smjera karakteristične za rešetku. Desno: ista putanja izgladena B-spline krivuljom s 200 ravnomjerno raspoređenih točaka. Scenarij `one_obstacle`, generiran stvarnim kodom iz `src/astar.py` i `src/path_smoother.py`.

4.3. Profil brzine

Na ravnim dionicama robot vozi punom brzinom $v_{max} = 1,0$ m/s. U zavojima je poželjno usporiti kako bi se smanjila bočna greška praćenja. Referentna brzina računa se iz zakrivljenosti κ putanje:

$$v_{ref}(s) = \frac{v_{max}}{1 + k_{curv} \cdot \kappa(s)}, \quad k_{curv} = 0,5 \quad (4.4)$$

gdje se zakrivljenost aproksimira vektorskim produktom uzastopnih segmenata putanje. Profil brzine koristi se isključivo za vizualizaciju referentnog signala u grafovima — NMPC sam optimizira stvarnu brzinu unutar zadanih ograničenja.



Slika 4.5. Profil brzine u scenariju L-hodnika. Isprekidana linija prikazuje referentnu brzinu v_{ref} izračunatu prema zakrivljenosti putanje — vidljivi su padovi na zavojima. Puna linija prikazuje stvarnu brzinu koju NMPC optimizira neovisno o v_{ref} .

5. Implementacija sustava

5.1. Arhitektura sustava

Sustav je implementiran u Pythonu i organiziran u module koji odgovaraju logičkim slojevima arhitekture. Svaki modul ima jasno definiran ulaz i izlaz bez globalnog stanja. Sve konfiguracijske vrijednosti — dimenzije okoline, parametri robota, horizonti i težine MPC-a — čitaju se iz jedne datoteke `config/params.yaml`, čime je odvojena konfiguracija od logike.

Izvršavanje počinje u `src/scenarios.py`, koji za odabrani scenarij vraća rječnik s početnim stanjem robota, koordinatama cilja i popisom prepreka. Prepreke su definirane u `src/obstacles.py` kao instance klasa `Circle` i `Rectangle`. Pravokutne prepreke se interno aproksimiraju mrežom malih kružnica kako bi bile kompatibilne s `acados`-ovim sustavom nelinearnih ograničenja koji očekuje analitičke izraze u obliku $h(\mathbf{x}) \geq 0$.

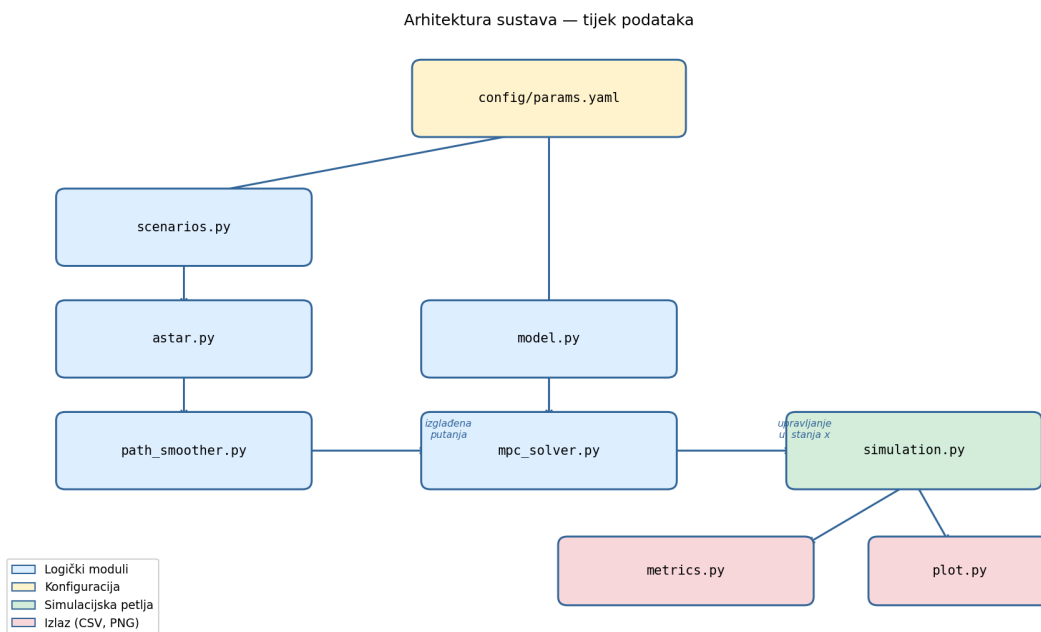
Na temelju popisa prepreka i pozicija starta i cilja, `src/astar.py` gradi diskretiziranu mrežu okoline rezolucije 0,2 m i markira sve čvorove koji su unutar infliranog radijusa prepreke kao zauzete. Algoritmom A* s euklidskom heuristikom i mogućnošću kretanja u osam smjerova pronalazi se kolizijsko-slobodna putanja u obliku niza čvorova mreže. Ako put ne postoji, algoritam baca iznimku i solver se ne pokreće.

Putanja dobivena A* algoritmom potom ulazi u `src/path_smoother.py`. Funkcija `smooth` uklanja duplikate točaka i aproksimira ih kubičnim B-splineom s parametrom glaćenja $s=0,5$, uzorkovanim na 200 ravnomjerno raspoređenih točaka. Funkcija `velocity_profile` zatim za svaku točku izgladene putanje procjenjuje zakrivljenost iz promjene smjera susjednih segmenata i postavlja referentnu brzinu obrnuto proporcionalnu zakrivljenosti — uski zavoji za potrebe vizualizacije dobivaju nižu prikazanu referentnu brzinu; NMPC autonomno bira stvarnu brzinu unutar $v_{min} \leq v \leq v_{max}$.

Paralelno s planiranjem putanje, `src/model.py` definira simbolički CasADi model monocikla koji se predaje modulu `src/mpc_solver.py`. Solver pri inicijalizaciji kreira `acados` [6] `AcadosOcp` objekt: postavlja matrice troška Q i R , intervalna ograničenja na stanja i upravljanje te, ako scenarij ima prepreke, dodaje nelinearna ograničenja za svaku prepreku kao meko ograničenje s kaznenim faktorom 10^4 . Iz tog opisa `acados` generira i kompajlira optimizirani C kod solvera koji se koristi u simulaciji.

Sama simulacija odvija se u `src/simulation.py` kao zatvorena petlja opisana u odjeljku 5.2. Nakon završetka simulacije, `src/metrics.py` iz zabilježenih stanja, upravljanja i vremena rješavanja izračunava RMSE, CTE, grešku smjera i CPU statistiku. Rezultati se vizualiziraju modulima `src/plot.py` i `src/animate.py`.

Tijek podataka kroz sustav prikazan je na slici 5.1.



Slika 5.1. Arhitektura sustava. Konfiguracija (`params.yaml`) napaja sve module parametrima. Lijevi stupac prikazuje put planiranja putanje, sredina solver, a desno simulacijsku petlju i izlaz.

5.2. Simulacijska petlja zatvorene petlje

Srž sustava je simulacijska petlja u `src/simulation.py`. Na svakom koraku:

1. pronade se najbliža sljedeća putna točka na putanji
2. postavi se referenca $\mathbf{y}_{ref}$ za svih N koraka horizonta

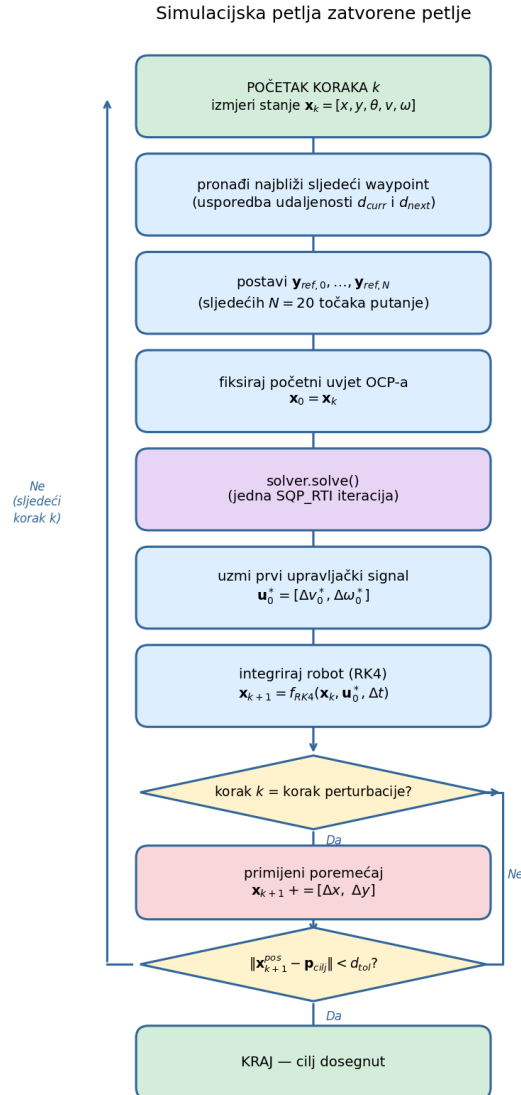
3. fiksira se trenutno stanje kao početni uvjet OCP-a
4. poziva se `solver.solve()` koji izvršava SQP_RTI iteraciju
5. uzima se prvi upravljački signal $\mathbf{u}_0^*$
6. integrira se model RK4 metodom
7. primjenjuje se eventualni poremećaj

```

1  for step in range(max_steps):
2      # napreduj do najbliže sljedeće putne točke
3      while wp_idx < len(waypoints) - 1:
4          if np.linalg.norm(x[:2] - waypoints[wp_idx+1]) \
5              <= np.linalg.norm(x[:2] - waypoints[wp_idx]):
6              wp_idx += 1
7          else:
8              break
9
10     # postavi referencu za N koraka unaprijed
11     for k in range(N):
12         ref_idx = min(wp_idx + k, len(waypoints) - 1)
13         xr, yr = waypoints[ref_idx] # koordinate ciljne putne
14         ↪ točke
15         dx = waypoints[min(ref_idx+1, len(waypoints)-1)][0] - xr
16         dy = waypoints[min(ref_idx+1, len(waypoints)-1)][1] - yr
17         theta_ref = np.arctan2(dy, dx) # smjer putanje u toj
18         ↪ točki
19         solver.set(k, "yref", np.array([xr, yr, theta_ref, 0.0, 0.0]))
20
21     # fiksiraj trenutno stanje kao početni uvjet
22     solver.set(0, "lbx", x)
23     solver.set(0, "ubx", x)
24
25     solver.solve() # jedna SQP_RTI iteracija
26     u = solver.get(0, "u") # uzmi prvi upravljački signal
27     x = _rk4_step(x, u, dt) # integriraj simulirani robot
28
29     if np.linalg.norm(x[:2] - goal) < tol:
30         break # cilj dosegnut

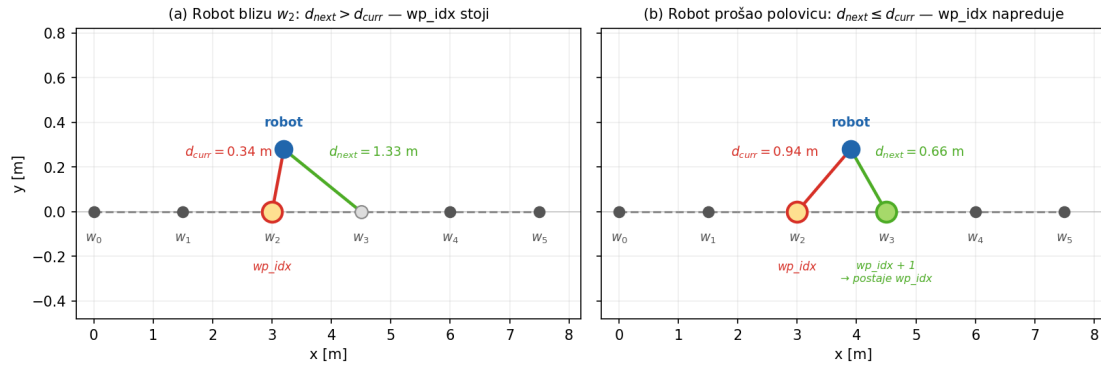
```

Listing 10: Jezgra simulacijske petlje u src/simulation.py



Slika 5.2. Dijagram toka simulacijske petlje zatvorene petlje. Ljubičasti blok označava SQP_RTI korak solvera, crveni blok primjenu poremećaja (prisutan samo u perturbacijskom scenariju).

Napredak po putanji određuje se usporedbom udaljenosti. Na svakom koraku algoritam izračunava $d_{curr} = \|\mathbf{p} - \mathbf{w}_{idx}\|$ i $d_{next} = \|\mathbf{p} - \mathbf{w}_{idx+1}\|$ te pomiče indeks naprijed sve dok vrijedi $d_{next} \leq d_{curr}$. Uvjet se provjerava u petlji pa robot može preskočiti više putnih točaka odjednom pri većim brzinama. Slika 5.3. ilustrira mehanizam: dok je robot blizu w_2 , vrijedi $d_{next} \gg d_{curr}$ i indeks stoji; čim robot prijeđe polovicu puta prema w_3 , sljedeća putna točka postaje bliža i indeks se pomiče. Na taj način referentna točka uvijek odgovara stvarnoj poziciji robota u prostoru.



Slika 5.3. Mehanizam napredovanja indeksa wp_idx . Lijevo: robot je blizu w_2 , vrijedi $d_{next} \gg d_{curr}$ i indeks stoji. Desno: robot je prešao polovicu segmenta prema w_3 , vrijedi $d_{next} \leq d_{curr}$ i indeks napreduje.

5.3. Metrike kvalitete praćenja

Modul `src/metrics.py` izračunava četiri mjere kvalitete iz rezultata simulacije.

RMSE (eng. *Root Mean Square Error*) mjeri prosječno odstupanje od referentne putanje:

$$RMSE = \sqrt{\frac{1}{K} \sum_{k=0}^{K-1} d_k^2}, \quad d_k = \min_j \|\mathbf{p}_k - \mathbf{w}_j\|_2 \quad (5.1)$$

gdje je d_k udaljenost od najbliže točke na putanji u koraku k . U implementaciji se za svaki vremenski korak izračunava udaljenost do svake točke izglađene putanje i uzima se minimum:

```

1  dists = np.linalg.norm(path - state[:2], axis=1)
2  idx   = np.argmin(dists)
3  d_k   = dists[idx]
```

Listing 11: Izračun udaljenosti d_k za RMSE u `src/metrics.py`

CTE (eng. *Cross-Track Error*) mjeri okomitu udaljenost od segmenta putanje — za razliku od RMSE-a uzima u obzir smjer putanje:

$$CTE_k = |(\mathbf{p}_k - \mathbf{w}_{j^*}) \times \hat{\mathbf{t}}_{j^*}| \quad (5.2)$$

gdje je $\hat{\mathbf{t}}_{j^*}$ smjer putanje u najbližoj točki w_{j^*} . Implementacija koristi skalarni oblik dvodimenzionalnog vektorskog produkta:

```

1  seg      = path[idx + 1] - path[idx]
2  seg_norm = seg / np.linalg.norm(seg)
3  diff     = state[:2] - path[idx]
4  cte      = abs(diff[0] * seg_norm[1] - diff[1] * seg_norm[0])

```

Listing 12: Izračun CTE u `src/metrics.py`

Greška smjera (eng. *heading error*) je apsolutna kutna razlika između orijentacije robota i smjera putanje, izračunata korištenjem `atan2` za ispravno rukovanje kutnim prelaskom:

$$e_{\theta,k} = \left| \text{atan2}(\sin(\theta_k - \theta_{ref,k}), \cos(\theta_k - \theta_{ref,k})) \right| \quad (5.3)$$

Direktno oduzimanje kutova $\theta_k - \theta_{ref,k}$ nije ispravno jer rezultat može izaći iz intervala $(-\pi, \pi]$; formulacija s funkcijom `atan2` osigurava da greška uvijek ostaje unutar tog intervala.

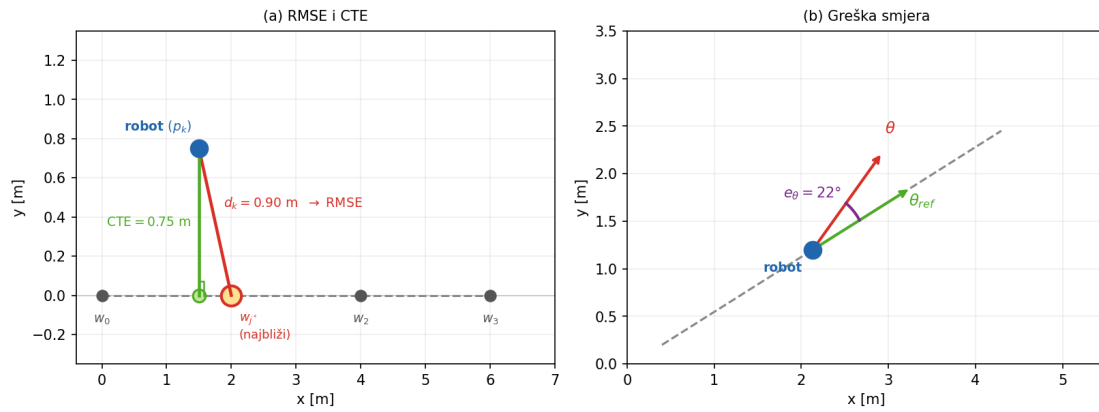
```

1  theta_ref = np.arctan2(seg[1], seg[0])
2  he = abs(np.arctan2(np.sin(state[2] - theta_ref),
3                      np.cos(state[2] - theta_ref)))

```

Listing 13: Izračun greške smjera u `src/metrics.py`

Geometrijska interpretacija sve tri mjere prikazana je na slici 5.4.



Slika 5.4. Geometrijska interpretacija mjera kvalitete. Lijevo: RMSE mjeri udaljenost do najbliže putne točke w_{j^*} (crvena linija d_k), dok CTE mjeri okomitu udaljenost do segmenta putanje (zeleni linija). Desno: greška smjera e_θ je kutna razlika između orijentacije robota θ i referentnog smjera putanje θ_{ref} .

CPU vrijeme mjeri se funkcijom `time.perf_counter()` neposredno oko svakog poziva `solver.solve()`, a bilježe se prosječno i maksimalno vrijeme po simulaciji:

```

1  t0 = time.perf_counter()
2  solver.solve()
3  solve_times.append(time.perf_counter() - t0)

```

Listing 14: Mjerenje CPU vremena jedne SQP_RTI iteracije u `src/simulation.py`

5.4. Testni scenariji

Sustav je testiran na osam scenarija različite složenosti: sedam u kojima robot uspješno dostiže cilj te jedan (`blocked`) koji testira prepoznavanje neizvedivog puta. U preostalim poglavljima pojam “sedam scenarija” odnosi se na ovih sedam navigacijskih scenarija, isključujući `blocked`. Tablica 5.1. prikazuje ključne karakteristike svih osam scenarija.

Tablica 5.1. Pregled testnih scenarija

Scenarij	Start	Cilj	Prepreke
baseline	(0,5; 0,5)	(9,5; 9,5)	—
one_obstacle	(0,5; 0,5)	(9,5; 9,5)	1 krug $r = 1,0$ m
narrow	(0,5; 5,0)	(9,5; 5,0)	2 kruga $r = 1,2$ m
l_corridor	(0,5; 1,0)	(9,0; 9,0)	1 pravokutnik
u_shape	(5,0; 0,5)	(5,0; 9,5)	3 pravokutnika
perturbation	(0,5; 5,0)	(9,5; 5,0)	— ($\Delta y = +1,0$ m u $t = 3$ s)
cluttered	(0,5; 0,5)	(9,5; 9,5)	7 krugova
blocked	(0,5; 5,0)	(9,5; 5,0)	5 krugova (zid)

Scenarij `blocked` je poseban — A^* ne može pronaći put pa se NMPC solver uopće ne pokreće. Sustav to prepoznaje i prikazuje kartu s porukom o neizvedivosti, što je opisano u poglavlju 6.

Svaki scenarij definiran je kao Python rječnik koji vraća funkcija u `src/scenarios.py`. Primjer scenarija s uskim prolazom između dviju kružnih prepreka:

```

1 def _narrow():
2     return {
3         "name": "narrow",
4         "start": np.array([0.5, 5.0, 0.0]),
5         "goal": np.array([9.5, 5.0]),
6         "obstacles": [
7             Circle(5.0, 3.5, 1.2),
8             Circle(5.0, 6.5, 1.2),
9         ],
10        "perturbation": None,
11    }

```

Listing 15: Definicija scenarija uskog prolaza u `src/scenarios.py`. Prepreke su instance klase `Circle(cx, cy, r)`; svi scenariji bez poremećaja imaju `perturbation: None`.

6. Rezultati i evaluacija

U ovom poglavlju prikazano je kako se sustav ponaša u praksi. Za svaki od sedam testnih scenarija prikazane su trajektorija robota, upravljački signali i greška praćenja, te su izmjereni pokazatelji kvalitete. Nakon toga analizirano je kako promjena dva ključna parametra sustava — predikcijskog horizonta N i težina troška — utječe na preciznost i brzinu izvođenja. Poglavlje završava tablicom usporedbe svih scenarija.

6.1. Eksperimentalni postav

Sva ispitivanja provedena su u simulacijskom okruženju implementiranom u Pythonu, korištenjem `acados` solvera za rješavanje OCP-a. Simulacijsko okruženje modelira ravnu površinu dimenzija $10\text{ m} \times 10\text{ m}$. Simulacija se odvija u zatvorenoj petlji s korakom uzorkovanja $\Delta t = 0,1\text{ s}$ i završava kada robot dođe unutar $d_{tol} = 0,3\text{ m}$ od cilja ili po isteku $k_{max} = 1000$ koraka. Metrike evaluacije definirane su u poglavlju 5., a pregled testnih scenarija u tablici 5.1.

Sva mjerenja izvršena su na prijenosnom računalu s procesorom Intel Core 7 150U i 32 GB RAM-a, pod operacijskim sustavom Fedora 42 (Linux kernel 6.19.14). Korišteni softver: Python 3.13.13, `acados` 0.5.1, `CasADi` 3.7.2, `SciPy` 1.17.1; solver je kompajliran kompajlerom GCC 15.2.1. CPU vrijeme jednog koraka mjereno je funkcijom `time.perf_counter()` neposredno oko poziva `solver.solve()` (prikazano u listingu u poglavlju 5.).

Tablica 6.1. prikazuje sve ključne parametre korištene u eksperimentima.

Tablica 6.1. Parametri simulacijskog okruženja

Skupina	Parametar	Oznaka	Vrijednost
Robot	Polumjer	r_{rob}	0,2 m
	Maksimalna brzina	v_{max}	1,0 m/s
	Minimalna brzina	v_{min}	0,05 m/s
	Maksimalna kutna brzina	ω_{max}	1,5 rad/s
	Maks. translacijsko ubrzanje	$a_{v,max}$	0,5 m/s ²
	Maks. kutno ubrzanje	α_{max}	1,0 rad/s ²
NMPC	Predikcijski horizont	N	20 koraka
	Korak uzorkovanja	Δt	0,1 s
	Težine pozicije	Q_x, Q_y	10,0
	Težina orijentacije	Q_θ	1,0
	Terminalna težinska matrica	W_e	Q
	Težine upravljanja	R_{a_v}, R_α	0,1
	Vrsta ograničenja prepreka	—	meka (soft)
	Vrsta solvera	—	SQP_RTI
A*	Rezolucija mreže	d_{grid}	0,2 m
	Ukupna inflacija prepreka	d_{inf}	0,4 m (0,2 sigurnost + 0,2 polumjer)
Simulacija	Tolerancija cilja	d_{tol}	0,3 m
	Integrator	—	RK4
	Maks. broj koraka	k_{max}	1000

Konfiguracija je pohranjena u datoteci `config/params.yaml`:

```

1  robot:
2    radius: 0.2
3    v_max: 1.0
4    v_min: 0.05
5    omega_max: 1.5
6    dv_max: 0.5
7    domega_max: 1.0
8  mpc:
9    N: 20
10   dt: 0.1
11   Q: [10.0, 10.0, 1.0]
12   R: [0.1, 0.1]
13   solver_type: SQP_RTI
14   constraints: soft
15 simulation:
16   goal_tolerance: 0.3
17   max_steps: 1000

```

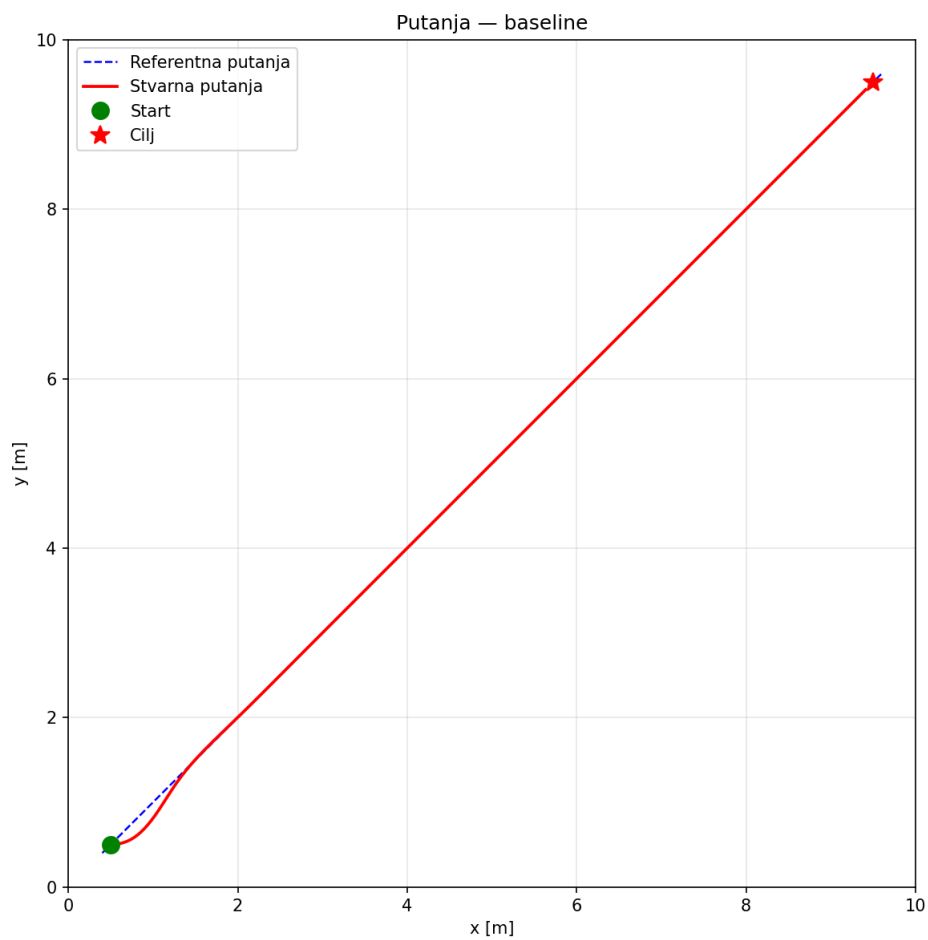
Listing 16: Ključni parametri u `config/params.yaml`

6.2. Osnovna putanja

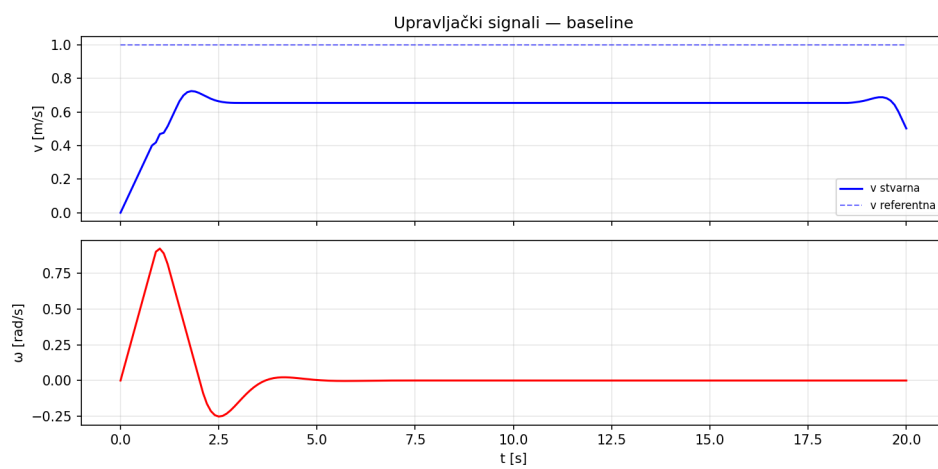
Scenarij osnovne putanje definira kretanje od (0,5; 0,5) s orijentacijom $\theta_0 = 0$ do cilja (9,5; 9,5), bez prepreka. Svrha je provjera ispravnosti sustava u idealnim uvjetima i uspostava referentnih vrijednosti za usporedbu s kompleksnijim scenarijima. A* planira dijagonalnu putanju koju algoritam glaćenja pretvara u glatku krivulju duljine oko 12,7 m.

Slika 6.1. pokazuje da stvarna putanja prati referentnu s RMSE od 34,4 mm. Jedino vidljivo odstupanje pojavljuje se na početku kretanja gdje robot s $\theta_0 = 0$ mora korigirati smjer prema dijagonalnom referentnom pravcu (kut $\approx 45^\circ$). NMPC regulator ispravno prepoznaje taj zahtjev i robota glatko preusmjerava bez oscilacija.

Graf upravljačkih signala (slika 6.2.) potvrđuje ispravno ponašanje: brzina v raste do $v_{max} = 1,0$ m/s i ostaje na toj razini do blizine cilja, dok kutna brzina ω pokazuje vrh od $\approx 0,87$ rad/s u prvoj sekundi (inicijalna korekcija smjera), a potom se stabilizira blizu



Slika 6.1. Referentna i stvarna putanja robota u scenariju bez prepreka.



Slika 6.2. Brzina $v(t)$ i kutna brzina $\omega(t)$ u scenariju bez prepreka.

nule.



Slika 6.3. Greška praćenja u funkciji vremena za scenarij bez prepreka.

Graf greške (slika 6.3.) prikazuje maksimum od $\approx 0,15$ m oko $t \approx 1,5$ s, što odgovara inicijalnoj korekciji smjera, a potom brzo opada ispod 0,03 m.

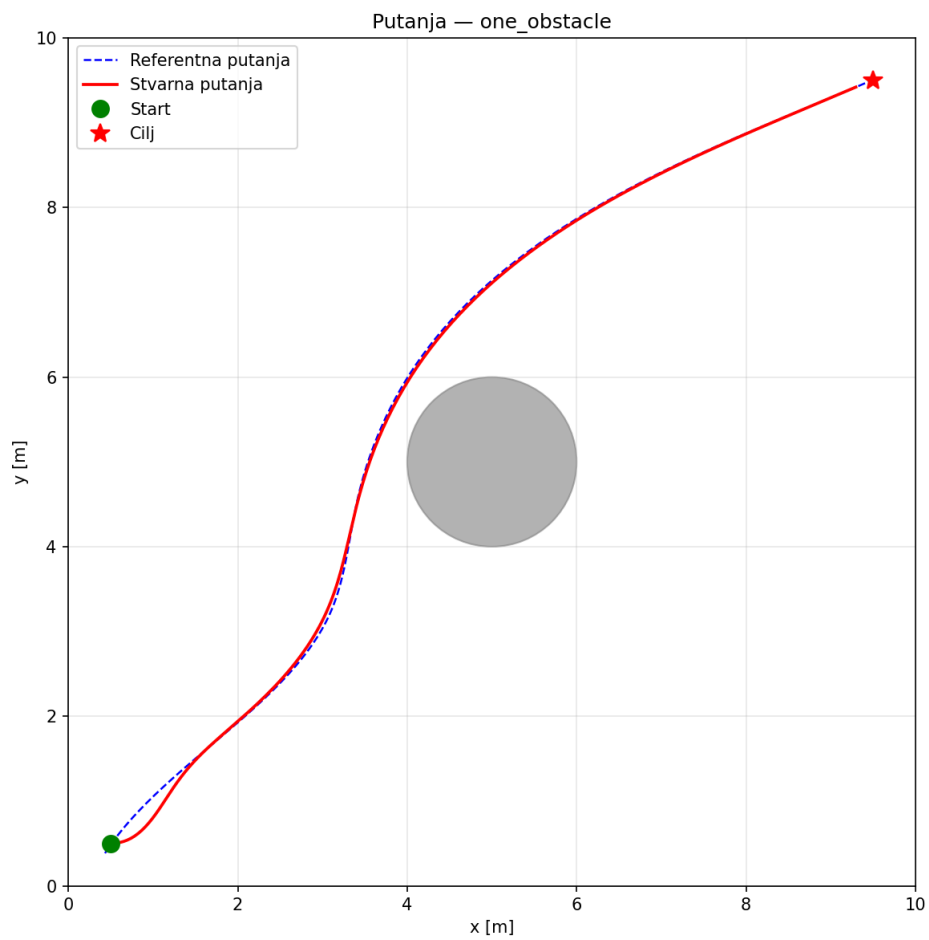
Tablica 6.2. Metrike — osnovna putanja

Metrika	Vrijednost
RMSE	0,0344 m
CTE srednji	0,0107 m
CTE max	0,1478 m
Greška smjera (srednja)	2,73°
Vrijeme do cilja	20,00 s
CPU (prosjek / max)	0,091 ms / 0,319 ms
Dostigao cilj	Da

RMSE od 34,4 mm i srednja bočna greška od 10,7 mm potvrđuju visoku kvalitetu praćenja. Prosječno CPU vrijeme od 0,091 ms po koraku iznosi manje od 0,1% trajanja uzornog intervala od 100 ms, što pokazuje da je sustav primjenjiv u stvarnim uvjetima gdje je vrijeme odziva ključno.

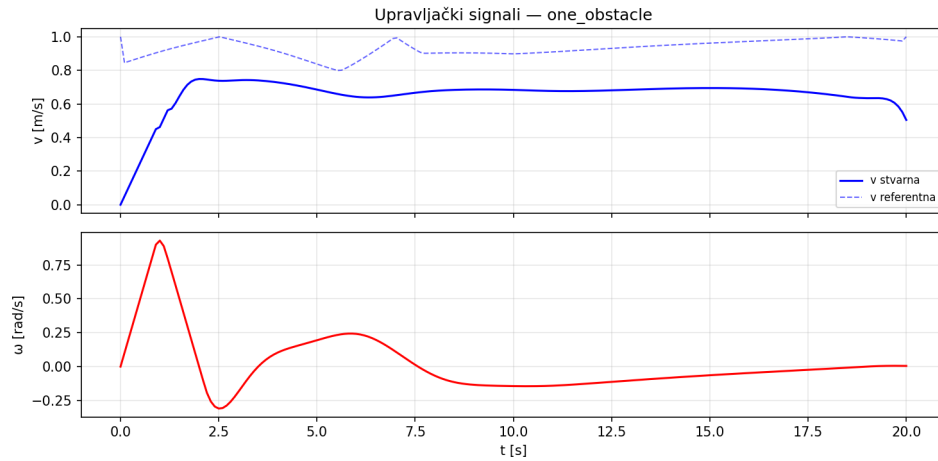
6.3. Jedna prepreka

Scenarij uvodi kružnu prepreku polumjera $r = 1,0$ m u središte prostora u točki (5,0; 5,0). Robot kreće iz (0,5; 0,5) prema cilju (9,5; 9,5), pa prepreka direktno blokira najkraću dijagonalnu putanju. A* pronalazi put koji zaobilazi prepreku s gornje lijeve strane.



Slika 6.4. Referentna i stvarna putanja u scenariju s jednom kružnom preprekom. Robot zaobilazi prepreku s gornje lijeve strane.

Slika 6.4. pokazuje da robot prati referentnu putanju s visokom točnošću, uključujući zavoj oko prepreke. Minimalna udaljenost od ruba prepreke iznosi više od 0,4 m, što je iznad zbroja polumjera robota i sigurnosne margine, pa meka ograničenja prepreka u NMPC-u nisu bila aktivna.



Slika 6.5. Brzina $v(t)$ i kutna brzina $\omega(t)$ u scenariju s jednom preprekom. Dva karakteristična vrha ω odgovaraju inicijalnoj korekciji smjera i zavoju oko prepreke.



Slika 6.6. Greška praćenja u scenariju s jednom preprekom.

Tablica 6.3. Metrike — jedna prepreka

Metrika	Vrijednost	Usporedba s osnovnim
RMSE	0,0469 m	+36,1%
CTE srednji	0,0278 m	+160,7%
CTE max	0,1839 m	+24,4%
Greška smjera (srednja)	3,87°	+42,0%
Vrijeme do cilja	20,00 s	0%
CPU (prosjek / max)	0,099 ms / 0,359 ms	+8,8%
Dostigao cilj	Da	—

RMSE i CTE su veći nego u osnovnom scenariju jer putanja sada ima zavoj oko prepreke — robot ne može trenutačno promijeniti smjer pa u krivini blago zaostaje za referencom. Unatoč tome, RMSE ostaje ispod 50 mm, a dodatno CPU opterećenje (+8,8%) je zanemarivo.

6.4. Uski prolaz

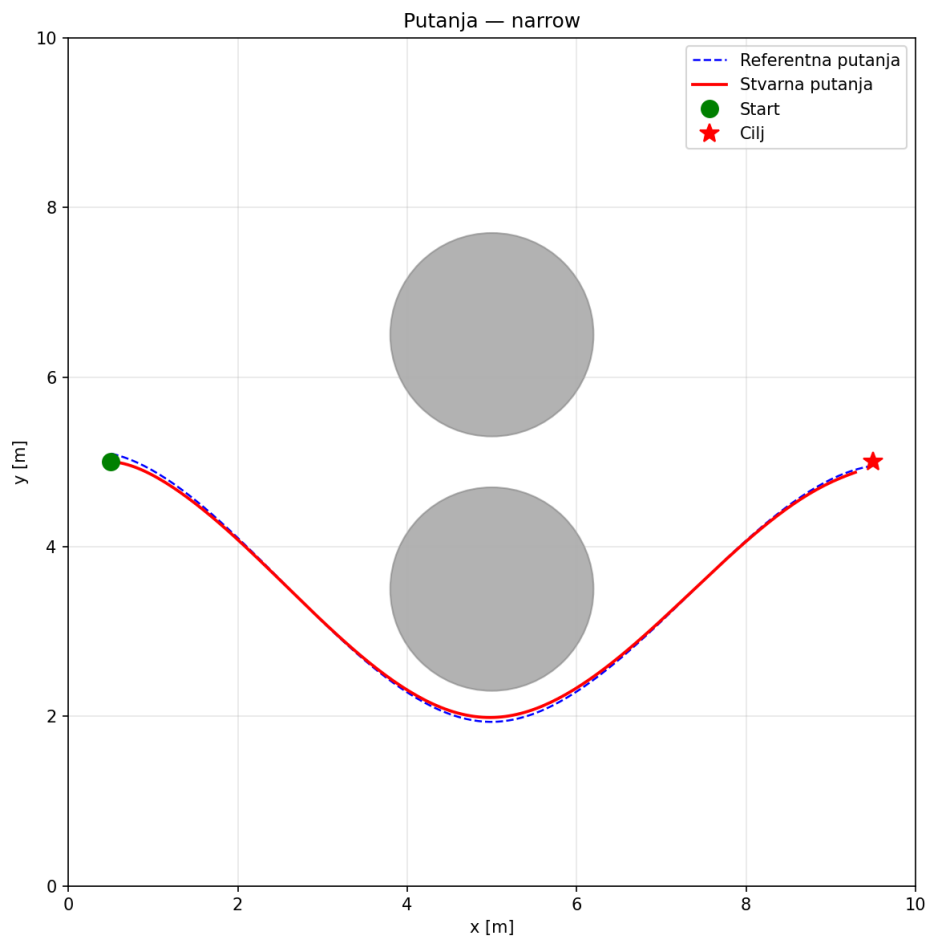
Scenarij postavlja dvije kružne prepreke polumjera $r = 1,2$ m u točkama (5,0; 3,5) i (5,0; 6,5). Robot kreće iz (0,5; 5,0) prema (9,5; 5,0), pa referentna putanja prolazi upravo između tih prepreka.

Geometrijska analiza otkriva ključnu karakteristiku scenarija: uzimajući u obzir ukupnu inflaciju od $d_{inf} = 0,4$ m (sigurnosna margina 0,2 m + polumjer robota 0,2 m), efektivna širina prolaza između infliranih prepreka iznosi:

$$d_{prolaz} = (6,5 - 1,2 - 0,4) - (3,5 + 1,2 + 0,4) = 4,9 - 5,1 = -0,2 \text{ m} \quad (6.1)$$

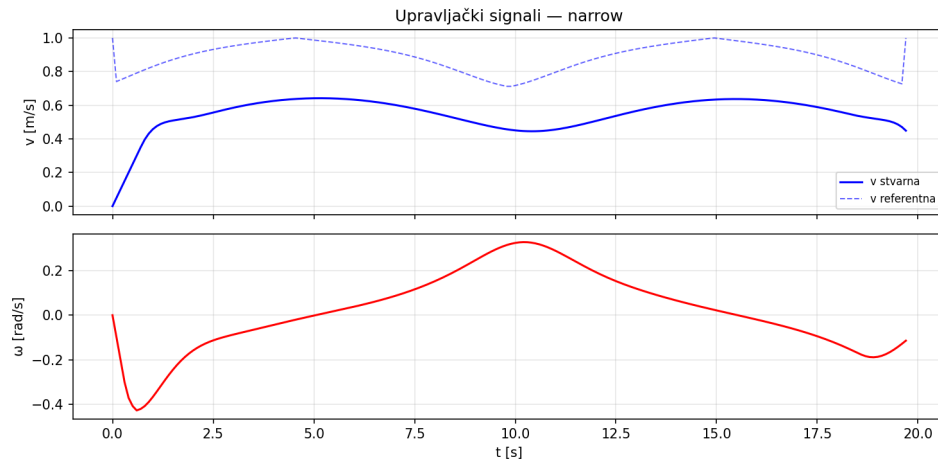
Negativna vrijednost znači da prolaz između prepreka nije prohodan za robota. A* zato planira put koji zaobilazi donju prepreku ispod nje, spuštajući putanju do $y \approx 2,2$ m.

Slika 6.7. prikazuje karakteristični luk: robot se spušta prema jugu, prolazi ispod



Slika 6.7. Referentna i stvarna putanja u scenariju uskog prolaza. A* planira put ispod donje prepreke jer je prolaz između njih blokiran proširenim područjem prepreka.

donje prepreke pri $y \approx 2,2$ m, a zatim se vraća prema ciljnoj koordinati $y = 5,0$ m. Robot prati zadanu putanju s visokom točnošću, a minimalna udaljenost od ruba donje prepreke iznosi oko 0,5 m.



Slika 6.8. Brzina $v(t)$ i kutna brzina $\omega(t)$ u scenariju uskog prolaza. Kontinuirani sinusoidalni profil ω odgovara zaobilaženju prepreke.



Slika 6.9. Greška praćenja u scenariju uskog prolaza. Dva područja povišene greške odgovaraju dvjema najizraženijim krivinama putanje.

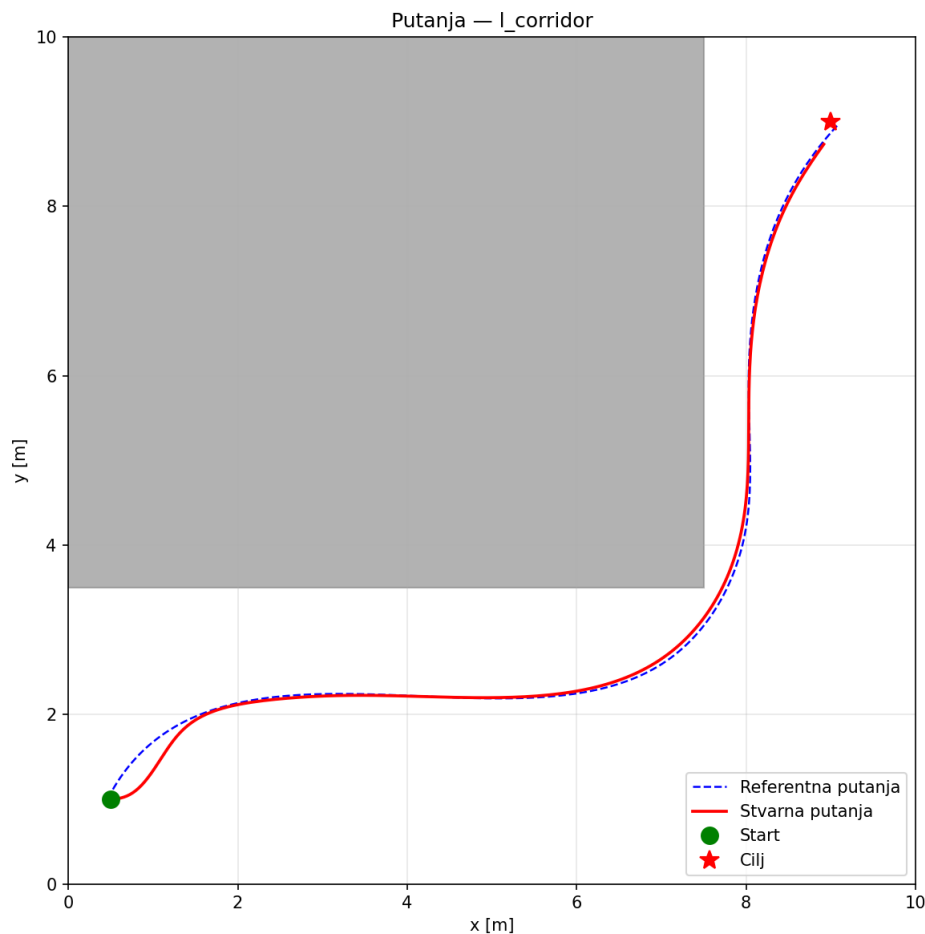
Tablica 6.4. Metrike — uski prolaz

Metrika	Vrijednost	Usporedba s osnovnim
RMSE	0,0344 m	$\approx 0\%$
CTE srednji	0,0270 m	+152,7%
CTE max	0,0962 m	−34,9%
Greška smjera (srednja)	1,59°	−41,5%
Vrijeme do cilja	19,70 s	−1,5%
CPU (prosjek / max)	0,089 ms / 0,263 ms	−2,2%
Dostigao cilj	Da	—

Ostvareni RMSE (34,4 mm) jednak je onom iz osnovnog scenarija, a greška smjera (1,59°) je najmanji od svih scenarija. Razlog je jednostavan: robot kreće s početnom orijentacijom $\theta_0 = 0$ (prema desno) i cilj se nalazi upravo u tom smjeru, pa ne treba inicijalna korekcija kuta — za razliku od osnovnog scenarija gdje je robot morao odmah skrenuti za $\approx 45^\circ$.

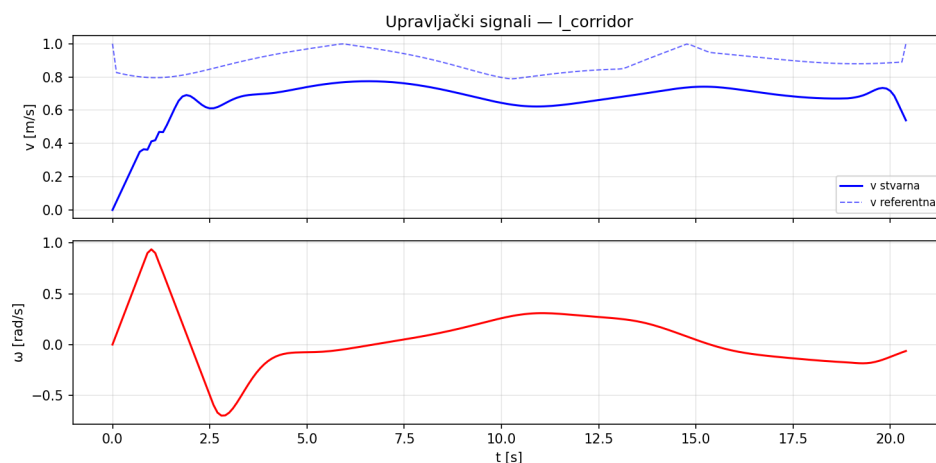
6.5. L-hodnik

U ovom scenariju velik pravokutni zid dimenzija $7,5 \text{ m} \times 6,5 \text{ m}$ blokira gornji lijevi dio prostora. Robot mora prijeći karakteristični L-oblik putanje: horizontalni prolaz duž donjeg ruba prepreke, a potom okret za 90° i vertikalni uspon uz njezin desni rub.



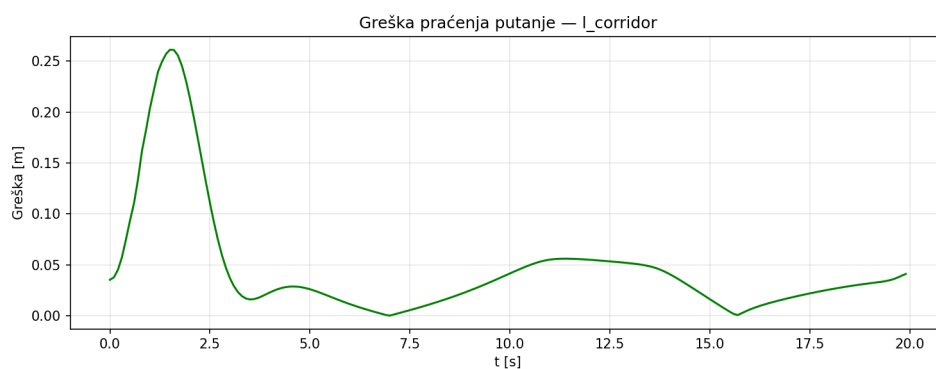
Slika 6.10. Referentna i stvarna putanja u scenariju L-hodnika. Pravokutna prepreka (siva površina) prisiljava robota na L-oblik putanje.

Kutna brzina dostiže $\omega_{min} \approx -1,0 \text{ rad/s}$, što je blizu ograničenja $\omega_{max} = 1,5 \text{ rad/s}$ i pokazuje da robot mora skrenuti gotovo maksimalnom kutnom brzinom kako bi prošao



Slika 6.11. Brzina $v(t)$ i kutna brzina $\omega(t)$ u scenariju L-hodnika. Negativni vrh $\omega \approx -1,0$ rad/s oko $t \approx 2,8$ s odgovara oštroj zaokretu na kraju horizontalnog dijela.

oštar ugao hodnika.



Slika 6.12. Greška praćenja u scenariju L-hodnika. Inicijalni vrh od 0,285 m najveći je u svim pravilnim scenarijima.

Tablica 6.5. Metrike — L-hodnik

Metrika	Vrijednost	Usporedba s osnovnim
RMSE	0,0724 m	+110,2%
CTE srednji	0,0457 m	+328,3%
CTE max	0,2610 m	+76,6%
Greška smjera (srednja)	4,90-	+79,7%
Vrijeme do cilja	20,40 s	+2,0%
CPU (prosjek / max)	0,103 ms / 0,251 ms	+13,2%
Dostigao cilj	Da	—

Najveći inicijalni vrh greške (0,285 m) nastaje kombinacijom greške pozicije i greške smjera na startu — robot stoji na $y = 1,0$ m s $\theta_0 = 0$, dok referentna putanja odmah skreće prema $y \approx 2,2$ m. Bez tog prijelaznog odziva prosječna greška u oba segmenta hodnika bila bi usporediva s jednostavnijim scenarijima.

6.6. U-oblik

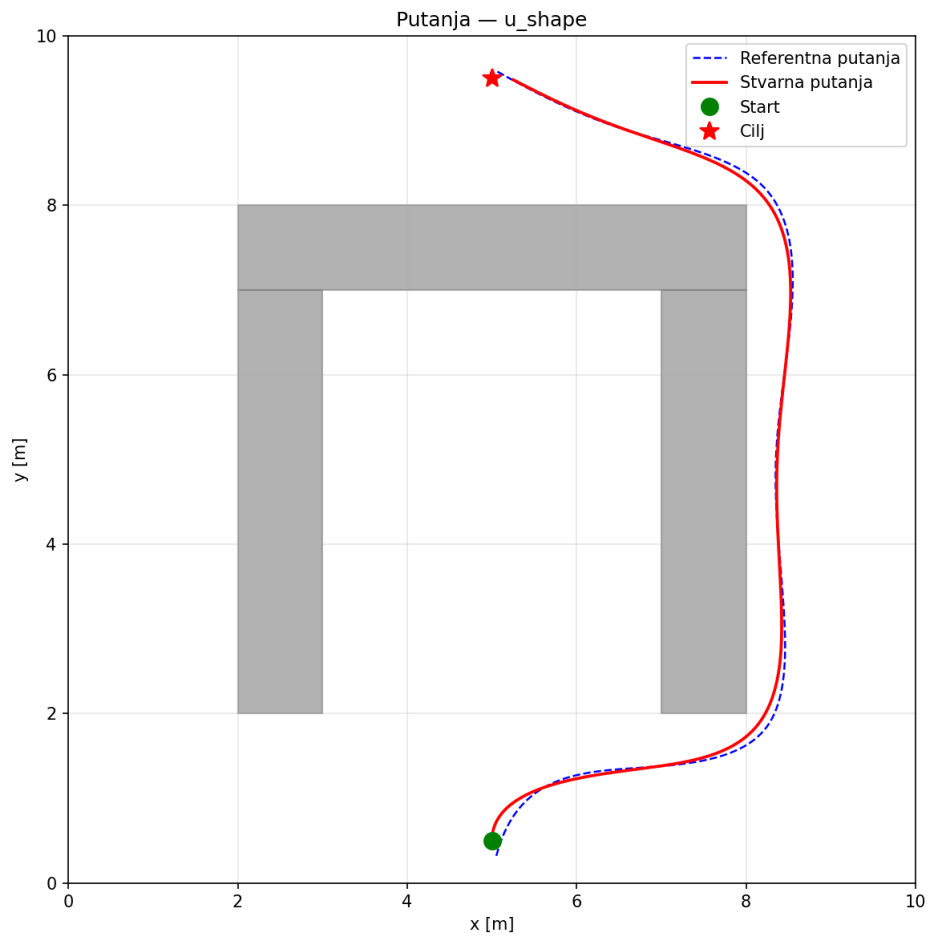
Tri pravokutne prepreke formiraju slijepu ulicu otvorenu prema jugu: lijeva stranica, desna stranica i gornja stranica U-oblika. Robot polazi iz (5,0; 0,5) s $\theta_0 = \pi/2$ i mora doseći cilj (5,0; 9,5), koji se nalazi izravno iza gornje stranice U. Naivni pristup prema cilju odveo bi robota ravno u zamku.

Scenarij demonstrira neophodnost dvoslojne arhitekture: A^* koji vidi cijeli prostor odjednom pronalazi jedini prohodani put — zaobilazeći U-oblik izvana s desne strane. NMPC s horizontom $N = 20$ (predikcija = 2 s) pak ne može sam zaključiti o topologiji zamke, pa bi bez globalne reference mogao zaglaviti unutar U-oblika.

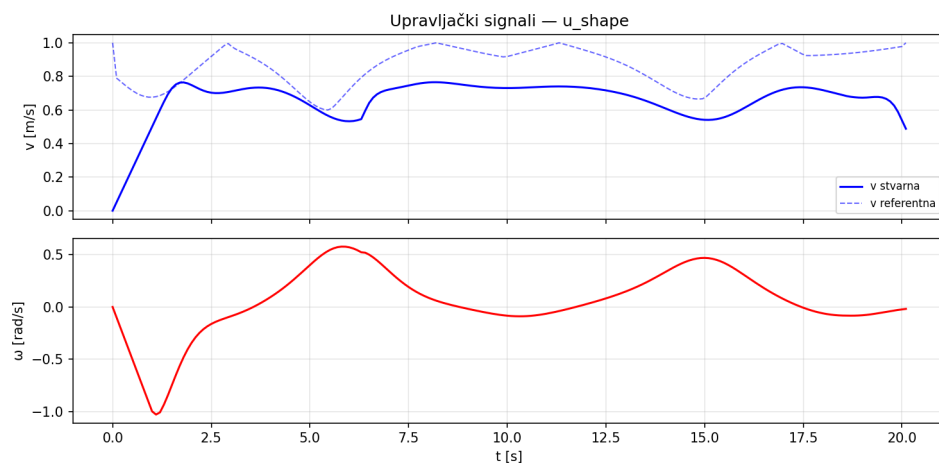
Tablica 6.6. Metrike — U-oblik

Metrika	Vrijednost	Usporedba s osnovnim
RMSE	0,0550 m	+59,8%
CTE srednji	0,0400 m	+274,5%
CTE max	0,1413 m	−4,4%
Greška smjera (srednja)	3,19°	+16,9%
Vrijeme do cilja	20,10 s	+0,5%
CPU (prosjeak / max)	0,114 ms / 0,270 ms	+25,3%
Dostigao cilj	Da	—

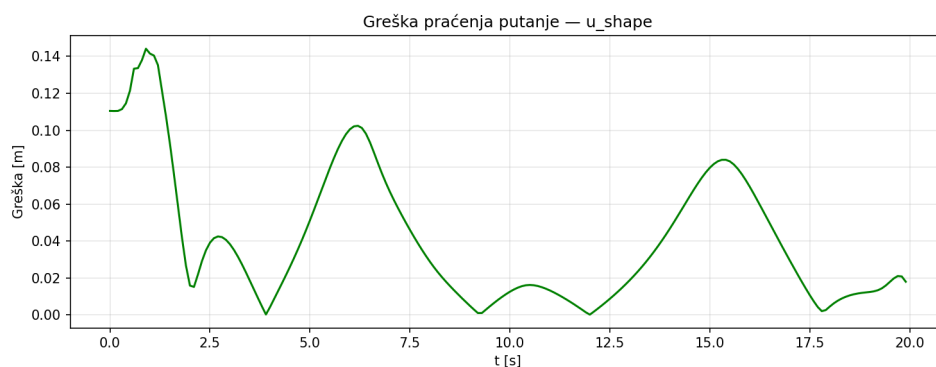
Uspješno dovršavanje ovog scenarija potvrđuje ispravnost dvoslojne arhitekture. RMSE od 55,0 mm drugi je po redu (iza L-hodnika) jer svaki od tri zavoja nakratko podigne grešku, no između zavoja praćenje je na razini osnovnog scenarija.



Slika 6.13. Referentna i stvarna putanja u scenariju U-oblika. A* planira put oko desne strane U-oblika, izbjegavajući slijepu ulicu.



Slika 6.14. Upravljački profili u scenariju U-oblika. Tri vrha ω odgovaraju trima zavojima na putu oko U-oblika.



Slika 6.15. Greška praćenja u scenariju U-oblika. Tri vrha greške odgovaraju trima zavojima putanje.

6.7. Poremećaj i oporavak

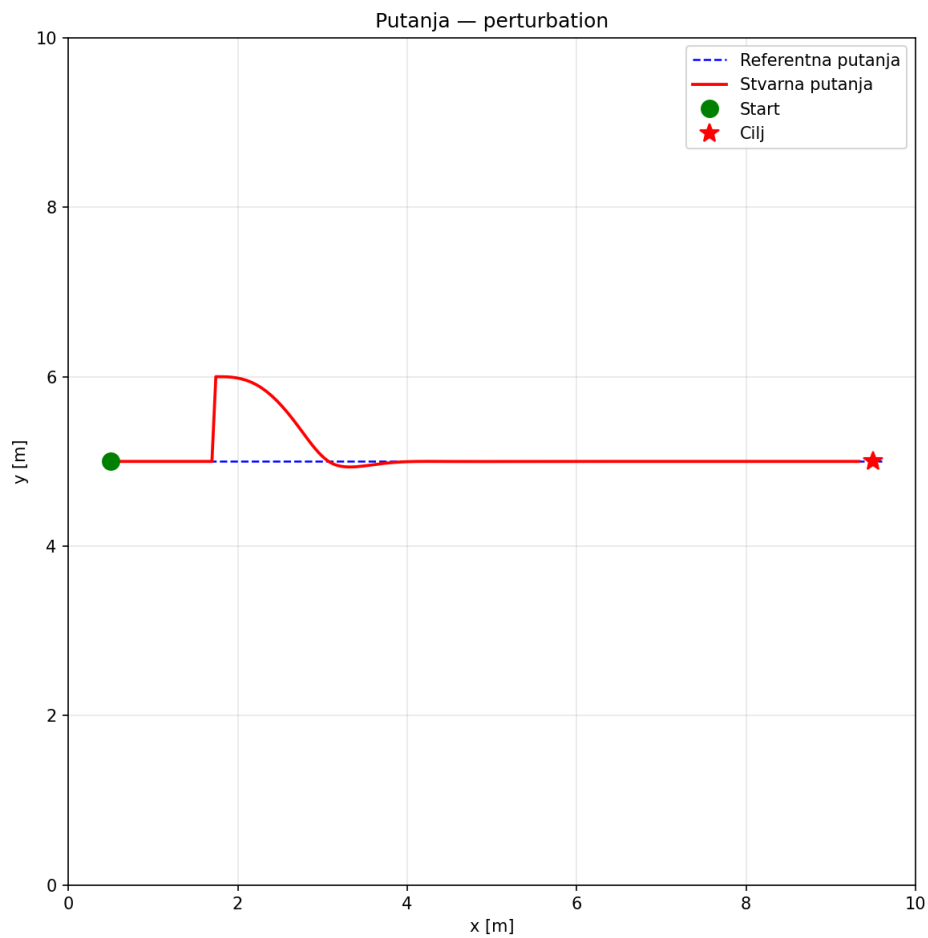
Scenarij ispituje sposobnost oporavka sustava od nagle vanjske sile. Konfiguracija je identična osnovnom scenariju (ravna putanja od (0,5; 5,0) do (9,5; 5,0)), s tom razlikom da se u koraku $k = 30$ ($t = 3,0$ s) stanje robota pomakne za $\Delta y = +1,0$ m. Poremećaj modelira nagli udarac — događaj koji sustav ne može predvidjeti jer nije dio modela.

Do $t = 3,0$ s robot savršeno prati horizontalnu referencu. U trenutku poremećaja robot je transliran na $y \approx 6,0$ m. NMPC u sljedećem koraku prima stvarno stanje, vidi grešku od 1,0 m i počinje ispravljati poziciju robota. Robot opisuje luk koji ga vraća prema referentnoj putanji i već pri $t \approx 5,2$ s ponovo se poravnava s referencom.

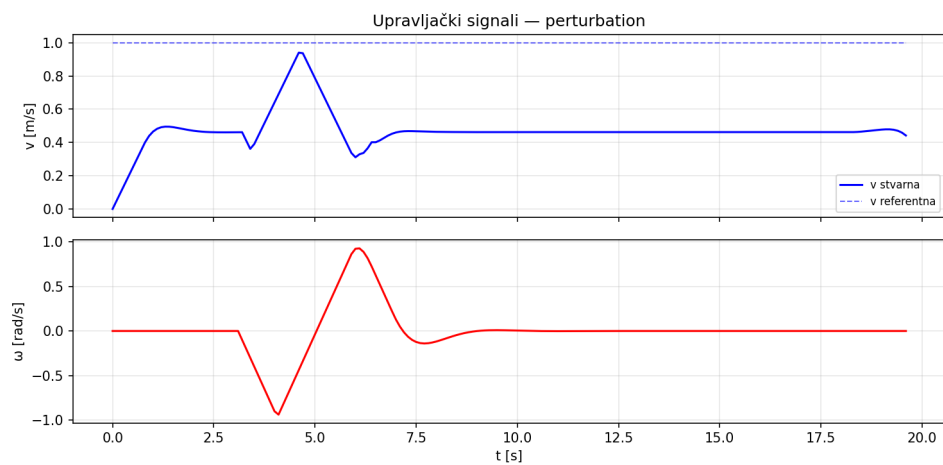
Tablica 6.7. Metrike — poremećaj

Metrika	Vrijednost
RMSE	0,2736 m
CTE srednji	0,0942 m
CTE max	1,0000 m
Greška smjera (srednja)	5,68°
Trajanje oporavka (do greške < 0,3 m)	22 koraka (2,2 s)
Vrijeme do cilja	19,60 s
CPU (prosjek / max)	0,118 ms / 0,321 ms
Dostigao cilj	Da

Visoki RMSE (273,6 mm) posljedica je isključivo jednog trenutačnog poremećaja.



Slika 6.16. Referentna i stvarna putanja u scenariju s poremećajem. Nagli pomak za 1,0 m vidljiv je kao karakteristični skok u trajektoriji, nakon kojega robot brzo vraća na referentnu putanju.



Slika 6.17. Upravljački signali u scenariju s poremećajem. Korektivni impulsi vidljivi su neposredno nakon $t = 3,0$ s.



Slika 6.18. Greška praćenja u scenariju s poremećajem. CTE skoči na 1,0 m pri $t = 3,0$ s i vraća se ispod 0,3 m unutar 22 koraka (2,2 s).

Ključni zaključak jest da NMPC u zatvorenoj petlji automatski ispravlja poremećaje — nije implementirana nikakva posebna logika za detekciju ili oporavak. Sustav na svakom koraku prima stvarno stanje i rješava OCP prema naprijed, što samo po sebi generira korektivne akcije.

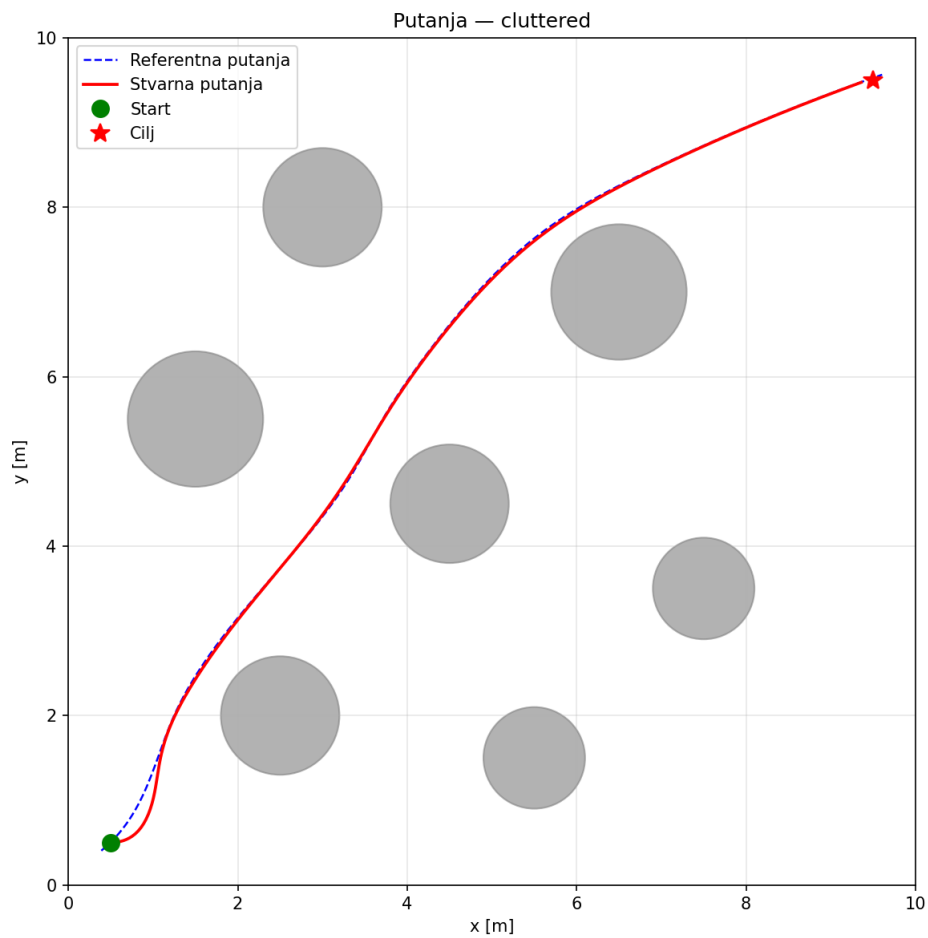
6.8. Gusti raspored prepreka

Scenarij smješta sedam kružnih prepreka polumjera 0,6–0,8 m na različite položaje u prostoru. Robot mora prijeći dijagonalnu putanju od (0,5; 0,5) do (9,5; 9,5). Scenarij testira skalabilnost sustava s povećanim brojem mekih ograničenja prepreka u OCP-u.

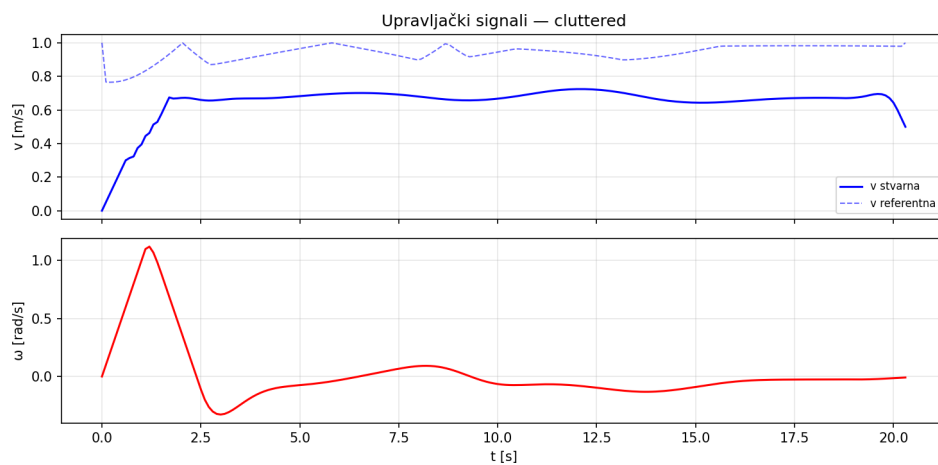
Stvarna brzina dosljedno se zadržava ispod $v_{max} = 1,0$ m/s jer NMPC autonomno bira nižu brzinu pri navigaciji kroz zakrivljenu putanju između sedam prepreka.

Tablica 6.8. Metrike — gusti raspored prepreka

Metrika	Vrijednost	Usporedba s osnovnim
RMSE	0,0429 m	+24,5%
CTE srednji	0,0232 m	+116,9%
CTE max	0,1678 m	+13,5%
Greška smjera (srednja)	3,43°	+25,8%
Broj prepreka	7	—
Vrijeme do cilja	20,30 s	+1,5%
CPU (prosjek / max)	0,107 ms / 0,277 ms	+17,6%
Dostigao cilj	Da	—



Slika 6.19. Referentna i stvarna putanja u scenariju gustog rasporeda prepreka. A* pronalazi glatku trajektoriju između svih sedam prepreka.



Slika 6.20. Upravljački profili u scenariju gustog rasporeda. Robot dosljedno održava brzinu oko 0,65–0,70 m/s — NMPC autonomno bira nižu brzinu pri navigaciji kroz zakrivljenu putanju.

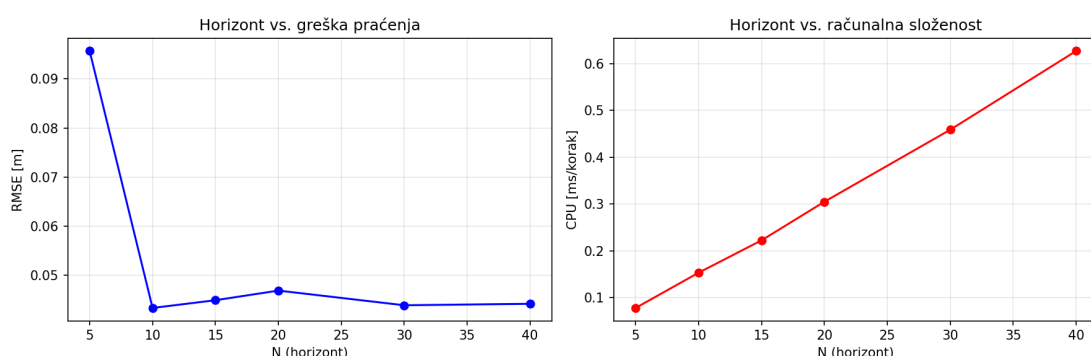


Slika 6.21. Greška praćenja u scenariju gustog rasporeda. Nakon inicijalnog vrha, greška ostaje u rasponu 0,01–0,033 m.

RMSE od 42,9 mm je iznimno dobar rezultat za scenarij s najvećim brojem prepreka — bolji od L-hodnika (72,4 mm) i U-oblika (55,0 mm). Razlog je glatka dijagonalna putanja bez naglih zavoja od 90°. Dodatno CPU opterećenje od 17,6% u odnosu na osnovni scenarij potvrđuje da dodavanje sedam mekih ograničenja nema značajan utjecaj na računalni teret.

6.9. Analiza osjetljivosti: predikcijski horizont N

Predikcijski horizont N temeljni je projektni parametar koji definira broj koraka unaprijed u OCP-u. Duži horizont teorijski omogućuje bolju predikciju, ali povećava dimenzionalnost optimizacijskog problema i time računalni trošak. Analiza je provedena za $N \in \{5, 10, 15, 20, 30, 40\}$ na scenariju s jednom preprekom.



Slika 6.22. Greška praćenja (RMSE) i prosječno CPU vrijeme u ovisnosti o predikcijskom horizontu N . Zadana vrijednost $N = 20$ označena je kao referentna.

Rezultati na slici 6.22. pokazuju tri zone:

- **Kratki horizont ($N = 5$):** RMSE iznosi 95,7 mm. Kratka predikcija ograničava

predviđanje zakrivljene putanje izbjegavanja prepreke.

- **Prijelazna zona** ($N = 10\text{--}20$): RMSE naglo pada na 43,3 mm pri $N = 10$, a zatim blago oscilira do 46,9 mm pri $N = 20$.
- **Zasićena zona** ($N = 30\text{--}40$): RMSE se stabilizira na ≈ 44 mm bez daljnjeg poboljšanja.

Tablica 6.9. Kompromis preciznosti i CPU-a za različite horizonte

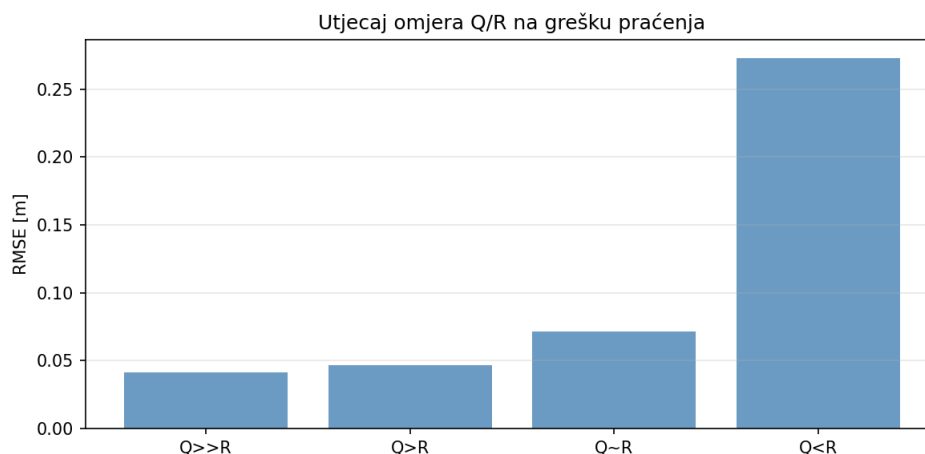
N	RMSE [mm]	CPU prosjek [ms]	Normalizirani CPU
5	95,7	0,078	1,0×
10	43,3	0,153	2,0×
15	44,9	0,222	2,9×
20	46,9	0,305	3,9×
30	43,9	0,459	5,9×
40	44,2	0,627	8,1×

CPU raste gotovo linearno s N , što je konzistentno s teorijskim ponašanjem SQP_RTI solvera čija složenost po iteraciji raste linearno s dimenzijom predikcijskog vektora. Odbir $N = 20$ temelji se na uravnoteženom kompromisu: u usporedbi s $N = 10$ postiže stabilniji RMSE, a u usporedbi s $N = 30$ razlika u RMSE manja je od 3 mm uz 33% manji CPU.

Napomena o metodološkoj razlici CPU vrijednosti. Vrijednosti CPU-a u tablici 6.9. (0,078–0,627 ms) veće su od vrijednosti u scenariju jedne prepreke (tablica 6.3., 0,099 ms za $N = 20$) iako je RMSE identičan (46,9 mm). Razlog je metodološka razlika: analiza horizonta pokrenuta je kao zaseban skript koji za svaki N iznova inicijalizira AcadosOcpSolver i generira novi C kod solvera. Svaka inicijalizacija uključuje hladan start (*cold start*) predikcijskog modela bez prethodno zagrijane JIT/kompajlerske predmemorije. Scenarijska usporedba koristi već kompajlirani solver, čiji su SQP_RTI pozivi brži zahvaljujući zagrijanju predmemorije instrukcija i podataka. Apsolutne vrijednosti u tablici 6.9. treba tumačiti kao gornju granicu CPU-a za hladni start pri promjeni konfiguracije, a ne kao radne vrijednosti u normalnom radu.

6.10. Analiza osjetljivosti: težine troška

Omjer Q/R temeljni je stupanj slobode koji određuje karakter NMPC-a: visoki Q kažnjava grešku praćenja, visoki R kažnjava agresivne promjene upravljanja. Testirana su četiri konfiguracijska profila uz konstantan $N = 20$:



Slika 6.23. RMSE praćenja za četiri konfiguracijska profila omjera Q/R u scenariju jedne prepreke.

Tablica 6.10. Utjecaj konfiguracije težina na preciznost praćenja

Konfiguracija	Q (pos.)	R	RMSE [mm]	Karakter upravljanja
$Q \gg R$	50,0	0,01	41,5	Intenzivno, responsivno
$Q > R$	10,0	0,1	46,9	Glatko, responsivno
$Q \sim R$	5,0	1,0	71,2	Slabije praćenje
$Q < R$	1,0	5,0	272,9	Pasivno, neučinkovito

Rezultati pokazuju monoton redoslijed: preciznost praćenja raste s porastom omjera Q/R . Konfiguracija $Q \gg R$ postiže najmanji RMSE (41,5 mm), no po cijenu agresivnijih upravljačkih akcija. Konfiguracija $Q < R$ ($Q = 1,0$, $R = 5,0$) postiže daleko najlošiji RMSE (272,9 mm) jer optimizator “bira” manje upravljačkih akcija kako bi smanjio trošak R , što rezultira trajnom velikom greškom pozicije.

Zadana konfiguracija $Q = 10,0$, $R = 0,1$ (omjer 100:1) odabrana je kao optimalno kompromisno rješenje koje postiže RMSE od 46,9 mm uz upravljačke signale dovoljno glatke za mehanički sustav.

6.11. Zbirna usporedba scenarija

Tablica 6.11. donosi objedinjeni pregled svih sedam testnih scenarija.

Tablica 6.11. Zbirna tablica rezultata — svi scenariji

Scenarij	RMSE [m]	CTE sred. [m]	Kut. greška [°]	Vrij. [s]	CPU prs. [ms]	Cilj
Osnovna putanja	0,0344	0,0107	2,73	20,0	0,091	Da
Jedna prepreka	0,0469	0,0278	3,87	20,0	0,099	Da
Uski prolaz	0,0344	0,0270	1,59	19,7	0,089	Da
L-hodnik	0,0724	0,0457	4,90	20,4	0,103	Da
U-oblik	0,0550	0,0400	3,19	20,1	0,114	Da
Poremećaj	0,2736	0,0942	5,68	19,6	0,118	Da
Gusti raspored	0,0429	0,0232	3,43	20,3	0,107	Da

Svi scenariji završavaju uspješnim dosezanjem cilja. Isključivanjem scenarija poremećaja, RMSE ostalih scenarija kreće se u rasponu od 34,4 mm do 72,4 mm (faktor 2,1×), što je izvrsna preciznost za sustav koji se kreće brzinom do 1 m/s.

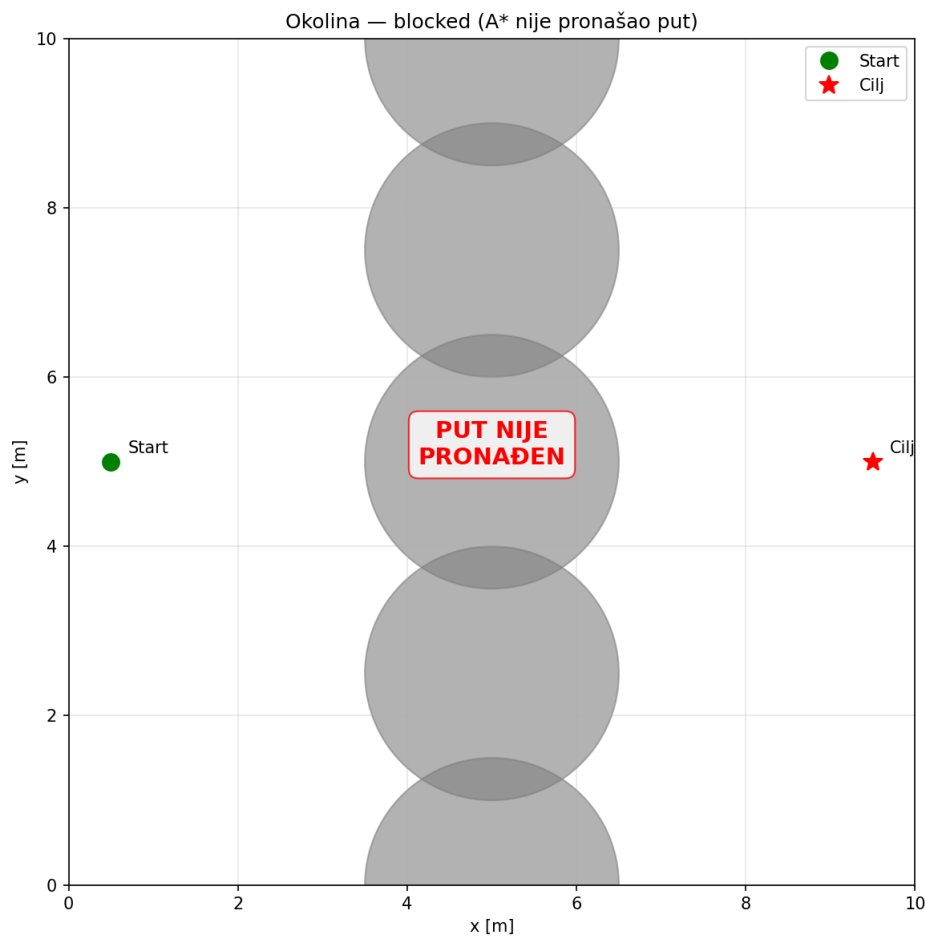
Prosječno CPU vrijeme iznosi između 0,089 ms i 0,118 ms, a maksimalni izmjereni korak traje 0,359 ms — što je oko 280× brže od raspoloživog uzornog intervala od 100 ms. Ovo eksperimentalno potvrđuje da je sustav primjenjiv u stvarnom vremenu i na hardveru s ograničenim procesorskim resursima. Budući da acados generira optimizirani C kod namijenjen ugradbenim sustavima, opravdano je očekivati prihvatljivo vrijeme izvođenja i na slabijem hardveru poput Raspberry Pi 4 — no to nije eksperimentalno verificirano u ovom radu i ostavlja se za budući rad (poglavlje 7.).

6.12. Nedostižni cilj — granice sustava

Scenarij blocked ispituje ponašanje sustava kada A* algoritam za planiranje putanje ne može pronaći putanju bez sudara s preprekama. Pet kružnih prepreka polumjera 1,5 m postavljeno je u vertikalni niz duž $x = 5,0$ m, formirajući potpunu prepreku od $y = 0$ do $y = 10$ m.

Efektivni polumjer svake prepreke pri A* pretraživanju iznosi $r + d_{inf} = 1,5 + 0,4 = 1,9$

m. Razmak između susjednih centara od 2,5 m manji je od $2 \times 1,9 = 3,8$ m, pa se inflirana područja potpuno preklapaju i formiraju neprobojni zid.



Slika 6.24. Vizualizacija blokirane okoline. Pet kružnih prepreka formira vertikalni zid koji potpuno blokira putanju od starta (lijevo) do cilja (desno).

A* ne pronalazi put i ispravno detektira neizvedivost — heuristika ga vodi sve dok ne iscrpi sve dostupne čvorove na strani starta:

```
1 RuntimeError: A*: nema puta od starta do cilja
```

Listing 17: Poruka sustava pri nedostižnom cilju

Budući da referentna putanja nije generirana, NMPC solver se uopće ne pokreće. Ovo je ispravno ponašanje: A* algoritam djeluje kao *filter izvedivosti* koji detektira neriješivost na razini planiranja, što je računalno učinkovito i arhitekturno ispravno.

7. Zaključak

7.1. Pregled postignutih rezultata

U ovom radu razvijen je dvoslojni sustav za planiranje i praćenje putanje mobilnog robota u poznatom statičkom prostoru. Globalni sloj temelji se na A* algoritmu koji pretražuje diskretiziranu mrežu rezolucije 0,2 m uz ukupnu inflaciju prepreka 0,4 m, a dobivena putanja se izgladuje uz vizualizacijski profil brzine prilagođen zakrivljenosti. Lokalni sloj koristi NMPC regulator s proširenim modelom monocikla i SQP_RTI solverom koji na svakom uzorkovnom koraku od $\Delta t = 0,1$ s rješava OCP s horizontom $N = 20$ koraka.

Sustav je testiran na sedam scenarija koji pokrivaju spektar od jednostavne ravne putanje do gusto popunjene okoline i namjerno zadanog poremećaja stanja. Svi scenariji završili su uspješnim dosezanjem cilja, s RMSE od 34,4 do 72,4 mm u statičkim scenarijima i prosječnim CPU vremenom od 0,089 do 0,118 ms po koraku.

7.2. Analiza ključnih nalaza

Preciznost praćenja i utjecaj geometrije okoline

Rezultati pokazuju da preciznost praćenja nije monotono određena brojem prepreka, nego geometrijom putanje koju prepreke nameću. Scenarij s jednom preprekom (RMSE = 46,9 mm) postiže lošiji rezultat od gustog rasporeda sedam prepreka (RMSE = 42,9 mm), jer raspoređene prepreke u potonjem slučaju dopuštaju glatku dijagonalnu trajektoriju, dok jedna prepreka u nepovoljnom položaju prisiljava na oštriji zaokret. Scenariji s hodnicima (L-hodnik, U-oblik) s RMSE od 72,4 mm i 55,0 mm postavljaju najveći zahtjev jer oblik hodnika nameće skretanja blizu 90°.

Automatski oporavak od poremećaja

Scenarij perturbacije pokazuje prednost MPC arhitekture: oporavak od poremećaja nije posebno kodiran, već je prirodna posljedica zatvorene optimizacijske petlje. Nagli pomak od $\Delta y = 1,0$ m kompenziran je unutar 2,2 s (22 koraka) bez ikakvih modifikacija sustava. Sustav na svakom koraku prima stvarno stanje i rješava OCP prema naprijed, pa automatski ispravlja grešku bez posebne logike za oporavak.

Računalna provedivost

Prosječno CPU vrijeme kreće se od 0,089 do 0,118 ms po koraku, što je daleko ispod uzornog intervala od 100 ms. Zanimljiv nalaz je da CPU prosjek ne ovisi izravno o broju prepreka: scenarij poremećaja bez prepreka postiže najviši prosjek (0,118 ms) zbog intenzivnih korektivnih iteracija, dok gusti raspored sedam prepreka iznosi 0,107 ms. Topologija putanje i dinamika stanja imaju veći utjecaj na računalni teret od broja mekih ograničenja.

Analiza osjetljivosti parametara

Analiza horizonta N otkriva zasićenje preciznosti za $N \geq 10$ ($\text{RMSE} \approx 43\text{--}47$ mm), dok CPU raste gotovo linearno. Odabrani $N = 20$ (0,305 ms/korak) postiže 33% manji CPU od $N = 30$ uz manje od 3 mm razlike u RMSE. Analiza težina Q/R pokazuje da što je Q veći u odnosu na R , robot preciznije prati putanju. Konfiguracija $Q < R$ rezultira najlošijim RMSE (272,9 mm) jer robot tada više kažnjava agresivno skretanje nego grešku pozicije.

7.3. Ograničenja implementiranog sustava

Implementirani sustav počiva na nekoliko pretpostavki koje ograničavaju izravnu primjenjivost u stvarnim scenarijima:

Poznata i statička okolina. A* algoritam zahtijeva potpunu kartu okoline unaprijed. U dinamičkim okolinama s pokretnim preprekama potrebna je integracija senzorskog sustava i reaktivno replaniranje.

Kinematički model bez dinamike. Model monocikla zanemaruje inerciju i trenje

kotača. Na visokim brzinama ili klizavim podlogama ova aproksimacija nije valjana i zahtijeva dinamički model.

Kružne prepreke. Implementirana ograničenja prepreka pretpostavljaju kružni oblik. Prepreke proizvoljnog oblika zahtijevaju konveksnu aproksimaciju poligona ili pristup temeljen na polju predznačenih udaljenosti (SDF).

Simulacijsko okruženje i identičan model. Svi rezultati dobiveni su u simulaciji. Dodatno, NMPC predikcijski model i model koji simulira stvarnog robota su identični — oba koriste isti kinematički model monocikla. Zbog toga NMPC "zna" točno kako će se robot ponašati, što daje gornju granicu preciznosti: izmjerene greške praćenja predstavljaju optimistični scenarij koji se na stvarnom robotu ne može dosegnuti. Na stvarnom hardveru prisutni su šum senzora, kašnjenje aktuatora i trenje koje uzrokuju odstupanje od predikcijskog modela, pa je stvarna preciznost praćenja nešto lošija.

7.4. Smjernice za buduća istraživanja

Na temelju postignutih rezultata i identificiranih ograničenja, predlažu se sljedeće smjernice za daljnji razvoj:

1. **Dinamičke prepreke i reaktivno replaniranje:** implementacija A* ponovnog planiranja putanje u stvarnom vremenu pri detekciji novih prepreka
2. **Dinamički model robota:** uključivanje inercije i trenja u model kako bi simulacija bila bliža stvarnom ponašanju robota
3. **Contouring MPC:** varijanta NMPC-a koja kažnjava bočno odstupanje i brzinu napredovanja po putanji umjesto praćenja diskretnih referentnih točaka
4. **Adaptivno podešavanje:** automatsko prilagođavanje težina Q i R ovisno o trenutnoj situaciji, npr. agresivnije u zavojima i glađe na ravnom dijelu putanje
5. **Testiranje na stvarnom hardveru:** pokretanje sustava na Raspberry Pi ili sličnom računalu kako bi se potvrdilo da radi dovoljno brzo u stvarnim uvjetima; pri tome bi trebalo implementirati i transformaciju $(v, \omega) \rightarrow (v_L, v_R)$ opisanu u odjeljku 2.4. za naredbu motornim kontrolerima diferencijalnog robota

7.5. Završna ocjena

Kombinacija A* algoritma i NMPC-a pokazala se kao dobro rješenje za navigaciju mobilnog robota — sustav precizno prati putanju, automatski se oporavlja od poremećaja i radi unutar zahtjeva intervala uzorkovanja od 100 ms s rezervom od dva reda veličine. Za razliku od jednostavnijih regulatora koji reagiraju tek nakon što se greška dogodi, NMPC gleda unaprijed i pokušava spriječiti grešku dok se još može.

Alat acados znatno pojednostavljuje implementaciju jer automatski generira efikasan solver iz zadane OCP formulacije, što oslobađa korisnika ručnog izvođenja derivacija i pisanja vlastitog solvera.

Literatura

- [1] P. E. Hart, N. J. Nilsson, i B. Raphael, “A formal basis for the heuristic determination of minimum cost paths”, *IEEE Transactions on Systems Science and Cybernetics*, sv. 4, br. 2, str. 100–107, 1968.
- [2] S. M. LaValle, *Planning Algorithms*. Cambridge, UK: Cambridge University Press, 2006.
- [3] R. C. Coulter, “Implementation of the pure pursuit path tracking algorithm”, Carnegie Mellon University, Robotics Institute, Pittsburgh, PA, teh. izv. CMU-RI-TR-92-01, 1992.
- [4] J. B. Rawlings, D. Q. Mayne, i M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2. izd. Madison, WI: Nob Hill Publishing, 2017.
- [5] F. Borrelli, A. Bemporad, i M. Morari, *Predictive Control for Linear and Hybrid Systems*. Cambridge, UK: Cambridge University Press, 2017.
- [6] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, i M. Diehl, “acados – a modular open-source framework for fast embedded optimal control”, *Mathematical Programming Computation*, sv. 14, br. 1, str. 147–183, 2022.
- [7] “acados documentation”, <https://docs.acados.org/>, 2024., pristupljeno: 2026.
- [8] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, i M. Diehl, “CasADi – a software framework for nonlinear optimization and optimal control”, *Mathematical Programming Computation*, sv. 11, br. 1, str. 1–36, 2019.

- [9] “CasADi — symbolic framework for numeric optimization”, <https://web.casadi.org/>, 2024., pristupljeno: 2026.
- [10] E. Hairer, S. P. Nørsett, i G. Wanner, *Solving Ordinary Differential Equations I: Nons-tiff Problems*, 2. izd., ser. Springer Series in Computational Mathematics. Berlin, Heidelberg: Springer, 1993., sv. 8.
- [11] B. Siciliano, L. Sciavicco, L. Villani, i G. Oriolo, *Robotics: Modelling, Planning and Control*, ser. Advanced Textbooks in Control and Signal Processing. London: Springer, 2009.
- [12] M. Diehl, H. G. Bock, J. P. Schlöder, R. Findeisen, Z. Nagy, i F. Allgöwer, “Real-time optimization and nonlinear model predictive control of processes governed by differential-algebraic equations”, *Journal of Process Control*, sv. 12, br. 4, str. 577–585, 2002.
- [13] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, Í. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, i SciPy 1.0 Contributors, “SciPy 1.0: fundamental algorithms for scientific computing in Python”, *Nature Methods*, sv. 17, str. 261–272, 2020.

Izjava o korištenju umjetne inteligencije

Tijekom izrade ovog završnog rada korišten je generativni AI alat Claude (Anthropic, verzija Claude Sonnet). Alat je korišten kao pomoć pri programiranju (debugiranje Python koda i konfiguracija acados solvera), istraživanju i razumijevanju literature te razumijevanju matematičkih pojmova vezanih uz NMPC i numeričku optimizaciju.

Osvrt na korištenje umjetne inteligencije: Implementacija algoritama (A*, NMPC formulacija u acadosu), odabir metodologije, provedba eksperimenata i interpretacija rezultata rezultat su mog samostalnog rada. Sve tvrdnje, matematičke izvode i zaključke provjerio sam na primarnim izvorima i provjerio kroz simulaciju.

Izjava o korištenju umjetne inteligencije: Potvrđujem da sam tijekom izrade ovog završnog rada koristio umjetnu inteligenciju u skladu s Politikom primjerenog korištenja umjetne inteligencije na Fakultetu elektrotehnike i računarstva, te da umjetna inteligencija nije zamijenila moje kritičko razmišljanje niti moj doprinos.

Sažetak

Planiranje putanje mobilnog robota u poznatom prostoru primjenom nelinearnog modelskog prediktivnog upravljanja

Luka Kordić

U radu je razvijen sustav za navigaciju mobilnog robota koji se kreće od zadane početne točke do cilja izbjegavajući prepreke. Sustav se sastoji od dva dijela: A* algoritma koji pronalazi slobodan put na mreži i NMPC regulatora koji taj put prati u stvarnom vremenu. Robot je modeliran kao monocikl, a putanja se izgladuje kako bi kretanje bilo glatko. NMPC u svakom koraku optimizira kretanje robota uzimajući u obzir ograničenja brzine i prepreke u okolini. Za implementaciju je korišten alat acados koji automatski generira optimizirani kod solvera. Sustav je testiran na sedam scenarija različite složenosti — od ravne putanje bez prepreka do gustog rasporeda sedam kružnih prepreka i scenarija s naglim bočnim guranjem robota. Svi scenariji završili su dosezanjem cilja. RMSE praćenja u statičkim scenarijima iznosi 34,4–72,4 mm; u scenariju s perturbacijom ($\Delta y = 1,0$ m) RMSE iznosi 273,6 mm, a robot se vraća na referencu unutar 2,2 s. Prosječno vrijeme izračuna po koraku iznosi 0,089–0,118 ms.

Ključne riječi: mobilni robot; NMPC; planiranje putanje; acados; SQP_RTI; A* algoritam; monocikl

Abstract

Mobile Robot Path Planning in a Known Environment Using Nonlinear Model Predictive Control

Luka Kordić

This thesis presents a navigation system for a mobile robot that moves from a start position to a goal while avoiding obstacles. The system consists of two parts: an A* algorithm that finds a free path on a grid, and an NMPC controller that follows the path in real time. The robot is modelled as a unicycle and the path is smoothed to ensure fluid motion. At each step, the NMPC optimises the robot's movement while respecting speed limits and obstacle constraints. The implementation uses the acados framework, which automatically generates optimised solver code. The system was tested on seven scenarios of varying difficulty — from a straight path with no obstacles to a cluttered environment with seven circular obstacles and a scenario with a sudden lateral push. All scenarios ended with the robot successfully reaching the goal. Path tracking RMSE in static scenarios ranges from 34.4 to 72.4 mm; in the perturbation scenario, where a 1.0 m lateral push is applied, RMSE reaches 273.6 mm with full recovery within 2.2 s. Average computation time per step is 0.089–0.118 ms.

Keywords: mobile robot; NMPC; path planning; acados; SQP_RTI; A* algorithm; unicycle